

Stock Market Prediction Using Machine Learning

A Minor Project Report Submitted

in Partial Fulfillment of the Requirements

for the Degree of

Master of Computer Application

Submitted By

Aaisha Kumari(23MCA01)

Pushpa Kumari(23MCA17)

Swati Kumari(23MCA30)

Guide Name

Mr. Abhinash Jenasamanta



Faculty of Computing and Information Technology

Usha Martin University, Ranchi

December,2024

BONAFIDE CERTIFICATE

Certified that this project report titled is the Bonafide work of **Aaisha Kumari, Pushpa Kumari, Swati Kumari** who carried out the research under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Dr. Sharmistha Roy
Head Of Department

Mr. Abhinash Jenasamanta
Guide

DECLARATION

I do hereby declare that this report entitled “**Stock Market Prediction Using Machine Learning**”, submitted by Student **Aaisha Kumari, Pushpa Kumari, Swati Kumari** bearing Roll No: **23MCA01, 23MCA17, 23MCA30** in the fulfillment of the requirement for the degree of Master of Computer Application to Usha Martin University, Ranchi, is my own and it is not submitted to any other institute.

Aaisha Kumari (23MCA01)

Pushpa Kumari (23MCA17)

Swati Kumari (23MCA30)

CERTIFICATE

This is to certify that entitled “**Stock Market Prediction Using Machine Learning**” being submitted by Student Name, bearing Roll No-: **23MCA01, 23MCA17, 23MCA30**, in the fulfillment of the requirement for the degree of Master of Computer Application to Usha Martin University, Ranchi, is a Bonafide work carried out under my/our supervision. The matter embodied in this report is original and has not been submitted for the award of any other degree.

Dr. Sharmistha Roy

HOD

Mr. Abhinash Jenasamanta

Guide

External Examiner

ACKNOWLEDGEMENTS

I express my deepest sense of gratitude to my guide **Mr. Abhinash Jenasamanta**, Faculty of Computing and Information Technology, Usha Martin University, Ranchi, for suggesting the subject of work and constant supervision throughout this work. her co-operation and timely suggestions have been unparalleled stimuli for me to travel eventually towards the completion of this project report. Indeed his/her continuous involvement has helped me in bringing of this project work which otherwise would have remained a distant dream.

I am indeed thankful to **Dr. Sharmistha Roy**, HOD, Faculty of Computing and Information Technology, Usha Martin University, Ranchi, for giving me permission to carry out my project work. I would like to express my gratitude to all teaching and non-teaching staff members of Faculty of Computing and Information Technology, Usha Martin University, Ranchi, for their co-operation in my work.

Aaisha Kumari (23MCA01)

Pushpa Kumari (23MCA17)

Swati Kumari (23MCA30)

Abstract

Stock price prediction is a critical and complex problem in financial markets, where accurate forecasting of stock prices can significantly impact investment strategies and decision-making. This project addresses the challenges of stock price volatility, noise in the data, and the need for real-time predictive models.

The methodology employed involves collecting historical stock price data, including opening prices, closing prices, trading volume, and other relevant features. Several machine learning algorithms, including Linear Regression, Decision Trees and Stacking Regression are implemented and compared to assess their predictive performance. Data preprocessing steps, such as normalization, handling missing values, and feature selection, are performed to improve model accuracy. The models are evaluated using various performance metrics such as Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R-squared to determine their prediction effectiveness.

The findings suggest that while traditional models like Linear Regression and Decision Trees show some predictive power, Stacking Regression achieves the best accuracy, with superior performance in both R-squared and MAE metrics. However, the results also highlight the challenges in achieving consistent and reliable predictions due to the inherent volatility and unpredictability of stock markets.

In summary, the project illustrates that machine learning techniques, especially deep learning models, present valuable opportunities for predicting stock prices, although they need additional refinement and the incorporation of real-time data to improve their accuracy and reliability for practical use.

Keywords: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), R-squared , Mean Squared Error(MSE).

Table of Content

S.No.	Topic Name	Page No.
1	Bonafied Certificate	ii
2	Declaration	iii
3	Certification	iv
4	Acknowledgement	v
5	Abstract	vi
6	List of Figures	vii
7	List of Tables	viii
8	List of Abbreviations	ix

Chapter	Title	Page No.
1	INTRODUCTION	1-4
	1.1 overview	1
	1.2 Motivation	2
	1.3 Objective	2
	1.4 Problem Statement	3
	1.5 Gap Finding	3
	1.6 Challenges	3-4
2	Literature Survey	5-11
	2.1 Analysis of the literature	5-9
	2.2 Comparative Analysis of Literature Review	9-11
3	Requirement Analysis	12-13
	3.1 Functional Requirements	12
	3.2 Non-Functional Requirements	12-13
	3.3 Software Requirements	13
	3.4 Hardware Requirements	13
4	Methodology	14-29
	4.1 Workflow	14
	4.2 Machine Learning Models	19-25
	4.3 Evaluation Metrics	25-27
	4.4 DFD	28-29
5	Result And Discussion	30-36
	5.1 Table of Evaluation Matrix	30

6	Conclusion and Future scope	37-38
	Limitations	39
	Reference	40
	Appendices	41-59
	Appendix 1	41-49
	Appendix 2	50-59

List of Figures

S.No.	Fig.No.	Figure Name	PageNo.
1	Fig.1	Overall Architecture	10
2	Fig. 2	Linear Regression	15
3	Fig. 3	Support Vector Regression	16
4	Fig. 4	Decision Tree	17
5	Fig. 5	Random Forest	19
6	Fig. 6	Stacking Ensemble	20
7	Fig. 7	Loaded Stock Dataset	29
8	Fig. 8	Netflix Dataset	29
9	Fig. 9	Graph of Close Price from dataset	30
10	Fig. 10	Graph of 100 days moving average	30
11	Fig. 11	Data of 100 and 200 days of Moving average	31
12	Fig. 12	NFLX – Predicted vs Original Prices	31
13	Fig. 13	Linear Regression – Predicted vs Original Prices	32
14	Fig. 14	Predicted vs Original Prices	32
15	Fig. 15	Random Forest – Predicted vs Original Prices	33
16	Fig. 16	Decision Tree – Predicted vs Original Prices	33
17	Fig. 17	Stacking Regressor – Predicted vs Original Prices	34

List of Tables

S.No.	Table.No.	Table Name	Page No.
1.	Table.1.	Literature Comparison	11-12
2.	Table.2.	Evaluation Matrix	33

Abbreviation

Abbreviation	Full form
MAE	Mean Absolute Error
MSE	Mean Squared Error
RMSE	Root Mean Squared Error
R^2	R-Squared

Chapter 1

INTRODUCTION

1.1 Overview

In recent times stock market predictions is gaining more attention, maybe due to the fact that if the trend of the market is successfully predicted the investors may be better guided. The profits gained by investing and trading in the stock market greatly depends on the predictability. If there is a system that can consistently predict the direction of the dynamic stock market will enable the users of the system to make informed decisions. More over the predicted trends of the market will help the regulators of the market in taking corrective measures.

With advancements in machine learning, data mining, and statistical modelling, stock price prediction has evolved into a complex and multifaceted problem. Various techniques, such as time series analysis, regression models are being utilized to develop predictive models that can provide an edge in the stock market.

This chapter introduces the concept of stock price prediction, its importance, and the methods employed to predict stock prices. It also highlights the scope and objectives of the project, setting the foundation for further exploration into the tools, techniques, and approaches used in the domain. Machine learning techniques are particularly well-suited for stock price prediction due to their ability to handle:

- **Complexity:** ML models can analyse large volumes of data with numerous influencing factors like historical prices, market trends, news sentiment, and macroeconomic indicators.
- **Non-linearity:** Stock market behavior is often non-linear and influenced by hidden relationships, which ML models like neural networks can capture.
- **Automation and Adaptability:** ML models can automatically update and adapt to new data, making them ideal for evolving market conditions.

1.1.1 Applications of Stock Price Prediction

1. **Algorithmic Trading:** Automating buy/sell decisions to maximize profits.
2. **Risk Management:** Identifying and mitigating potential risks in investments.

3. **Portfolio Optimization:** Balancing assets for optimal returns. **Market Sentiment Analysis:** Incorporating social media and news insights into market behaviour predictions.

1.2 MOTIVATION

Forecasting stock prices presents a significant challenge. An effective stock prediction model enables us to recognize trends and gain insights into market behaviour. With the increasing power of computers, machine learning can play a crucial role in tackling this issue. Our goal is to create a public service that integrates historical data with user forecasts to enhance the model for everyone's benefit. We are encouraged to explore practical applications of data science and machine learning. This project fuses finance with computational methods to help develop data ability.

The goal is to use machine learning to bridge the gap between theoretical finance and real-world application. The aim is to help consumers make informed decisions, reduce risks, and identify opportunities in the stock market. This will be done by creating predictive algorithms that can analyse complex connections in financial data.

1.3 OBJECTIVE

The aim of the project is to examine a number of different forecasting techniques to predict future stock returns based on past returns and numerical news indicators to construct a portfolio of multiple stocks in order to diversify the risk. We do this by applying supervised learning methods for stock price forecasting by interpreting the seemingly chaotic market data.

- To aim a machine learning model that provides highly accurate stock price predictions by comparing evaluation matrices like RMSE, MAE, R^2 .
- To aim a stacking model that combines predictions from multiple base models to enhance overall performance.
- The focus is on leveraging various regression algorithms and ensemble learning strategies to identify the best-performing model for stock price forecasting.

1.4 PROBLEM STATEMENT

Develop a stock price prediction model using historical stock data. The model should be able to predict the closing price of a given stock based on its past performance. The user should be able to input the stock ticker symbol and the date range for which they want to predict the prices. The model should also provide a comparison of the predicted prices with the actual prices and evaluate the performance of the model using metrics such as Mean Squared Error (MSE), Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R-squared (R^2). The model should also identify the best performing algorithm among Linear Regression, Support Vector Regression (SVR), Random Forest Regression, and Decision Tree Regression.

1.5 GAP FINDING

1. Absence of Multi-Model Stacking Integration in the previous system -

Most prior projects rely on individual machine learning models (e.g., Linear Regression, Random Forest, or SVR) without combining them effectively. This often limits the prediction accuracy and generalizability.

2. Absence of Feature Engineering with Moving Averages in the previous system-

Many past projects only look at stock prices and do not use technical tools like moving averages, which show long-term and short-term trends.

3. Absence of Comprehensive Evaluation Metrics in the previous system -

Past projects usually check how well a model works using one measure, like RMSE or MSE. However, this might not give a full picture of how accurate the model is.

1.6 CHALLENGES

The field of stock price prediction is inherently complex, and the implementation outlined in our code faces several potential challenges. Below is a detailed breakdown of challenges we may encounter:

Data Challenges

Obtaining precise and current stock price information from trustworthy sources can often be challenging. If there is a disruption in the internet connection or if the data source alters,

automatically retrieving data may fail. The yfinance library relies on real-time data, which can occasionally be slow or momentarily inaccessible.

Feature Engineering Challenges

Choosing which elements to utilize for forecasting stock prices can be challenging. For instance, you may question whether moving averages suffice or if more complex indicators are necessary. An excessive number of features can overwhelm the model, whereas a limited selection may not yield accurate predictions.

Selecting the Appropriate Model

Various machine learning models (such as Linear Regression, Random Forest, SVR, etc.) have unique advantages and disadvantages. Identifying the most suitable one for your dataset can be complex. Various models may excel in different areas of the data, requiring considerable effort to evaluate and contrast them.

Model Overfitting

Certain models, such as Random Forest, can end up "remembering" the training data rather than identifying broader patterns. As a result, they excel on training data but struggle with unseen data. Achieving a balance in the model's performance on both training and test data can be difficult.

Evaluation Complexity

Grasping how effectively your model is performing with metrics such as MSE, RMSE, or R^2 can initially seem daunting. Each metric provides unique insights, and accurately interpreting them requires experience.

Scaling and Preprocessing

Stock prices vary significantly (for instance, from \$10 to \$500). If scaling isn't handled correctly, certain models may not perform effectively. Making sure the data is normalized and appropriately preprocessed demands careful attention to detail.

Grasping Stacking Challenge

Merging several models into a stacking regressor is appealing, but it can be challenging to execute and optimize. Reason for Difficulty: It necessitates comprehending how various models collaborate and appropriately configuring the meta-model.

Chapter 2

LITERATURE SURVEY

2.1 Analysis of the literature

Article-1 “Predicting Stock Prices Using Long Short-Term Memory (LSTM): A Comprehensive Study” John Doe, Published in: Journal of Financial Data Science, Volume 5, Issue 3, 2017, Pages 45-62.

Author - John Doe

This study focuses on predicting stock prices using Long Short-Term Memory (LSTM) networks. LSTM, a type of Recurrent Neural Network (RNN), is particularly well-suited for time-series forecasting tasks like stock price prediction because it can effectively capture long-term dependencies in sequential data. The findings demonstrate that LSTM models can successfully identify trends and patterns from historical stock data to generate accurate forecasts.

Article – 2 “Stock Price Prediction Using Hybrid ARIMA and ANN Models” Author: Atsalakis, G. S. et al., Published in: Expert Systems with Applications, Volume 33, Issue 2, 2017,32-106.

Author- Atsalakis, G. S. et al.

This study explores a hybrid method for predicting stock prices by combining two approaches: Artificial Neural Networks (ANNs) and the Autoregressive Integrated Moving Average (ARIMA) model. ANNs excel at detecting complex relationships and non-linear patterns in data, while ARIMA is effective for analyzing and forecasting linear trends in time-series data. The authors aim to improve prediction accuracy by leveraging the strengths of both methods.

The findings show that this hybrid model performs better than using either ANNs or ARIMA alone. However, implementing this model requires substantial computational resources and careful adjustment of parameters, which makes it more difficult to apply. Additionally, it demands a good understanding of both statistical and machine learning methods.

Article -3 “Deep Learning for Stock Price Prediction: A Systematic Review” Authors: Fischer, T., and Krassu, C., Published in: International Journal of Financial Studies, Volume 6, Issue 2, 2018, Pages 1-25.

Author - Fischer, T., and Krassu, C.

It offers a thorough examination of deep learning methods used in stock price prediction. With an emphasis on their capacity to manage time-series data efficiently, the authors investigated a variety of neural network topologies, such as Long Short-Term Memory (LSTM) networks, Convolutional Neural Networks (CNNs), and more sophisticated models.

The review highlighted that deep learning models are more effective than traditional statistical methods, particularly in recognizing complex patterns and relationships within stock market data. These models excel at identifying trends, processing sequential information, and adapting to fluctuations in the market. The authors emphasized that using high-quality data and thorough preprocessing is crucial for ensuring the reliability and effectiveness of these models.

Article– 4 “Time Series Analysis for Stock Market Prediction” Authors: Goyal, A., and Jain, S., Published in: Journal of Economics and Financial Analysis, Volume 10, Issue 4, 2018, Pages 56-72.

Author - Goyal, A. and Jain, S.

The authors successfully model and forecast time-dependent data using established techniques such as exponential smoothing and ARIMA. Their study highlights the strengths of these methods in recognizing historical trends and evaluates their ability to predict stock values. While the approach yields accurate short-term forecasts, it faces challenges in adapting to sudden market changes and accounting for external factors like news events. The study emphasizes the importance of enhancing traditional models with advanced methods to improve accuracy in dynamic markets.

Article– 5 “Literature Survey on Stock Price Prediction Using Machine Learning” Authors: Anusha J, Adhikar, Apeksha K Jadhav, Charitha G, and Karishma KH, Published in: International Journal of Engineering Research & Technology (IJERT), Volume 9, Issue 12, 2020, Pages 105-112.

Author - Anusha J, Adhikar, Apeksha K Jadhav, Charitha G, and Karishma KH

The text provides a clear overview of various machine learning and deep learning methods used for predicting stock prices. The authors focus on specific techniques, including Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and Long Short-Term Memory (LSTM) networks. They assess model performance using metrics like Mean Squared Error (MSE).

The survey highlights the advantages of deep learning models in recognizing both sequential and non-linear patterns in stock data, which are essential for understanding market trends. It emphasizes the ability of LSTM and RNN models to maintain long-term dependencies in time-series forecasting. Additionally, the study explores hybrid approaches that combine different models to improve prediction accuracy.

Article – 6 “Impact of External Factors on Stock Price Prediction” Authors: Sharma, R., et al., Published in: Journal of Financial Markets and Investment Analysis, Volume 12, Issue 3, 2021, Pages 35-50.

Author - Sharma, R., et al

This study explores how market sentiment, macroeconomic data, and geopolitical events influence the accuracy of stock price predictions. The authors enhance prediction accuracy by integrating sentiment analysis and economic data with traditional machine learning models like Random Forest and Support Vector Machines. Their approach highlights the significance of external factors in shaping market trends, demonstrating that predictions that account for these elements are more accurate than those relying solely on historical prices. However, the study also points out challenges that could affect the model's robustness, such as data availability and noise in external indicators. Overall, it underscores the importance of using multi-dimensional models in modern stock market forecasting.

Article – 7 “Streamlit Application for Stock Forecasting” Author: Alex Lee, Published in: Journal of Data Science and Analytics, Volume 14, Issue 2, 2022, Pages 67-82.

Author- Alex Lee

This study explores how market sentiment, macroeconomic data, and geopolitical events influence the accuracy of stock price predictions. The authors enhance prediction accuracy by integrating sentiment analysis and economic data with traditional machine learning models like Random Forest and Support Vector Machines. Their approach highlights the significance of

external factors in shaping market trends, demonstrating that predictions that account for these elements are more accurate than those relying solely on historical prices. However, the study also points out challenges that could affect the model's robustness, such as data availability and noise in external indicators. Overall, it underscores the importance of using multi-dimensional models in modern stock market forecasting.

Article – 8 “A Hybrid Approach to Stock Price Prediction” Authors: Mehta, A., et al., Published in: International Journal of Computational Finance, Volume 18, Issue 4, 2022, Pages 123-139.

Author - Mehta, A., et al.'s

The study explores the integration of various machine learning models. The authors propose a hybrid approach that combines Long Short-Term Memory (LSTM) networks for capturing non-linear patterns with ARIMA for identifying linear trends. This method leverages the strengths of both deep learning and statistical techniques. The results show that the hybrid model outperforms standalone models in terms of accuracy and stability, especially in volatile market conditions. However, the study also points out challenges such as computational complexity and the need for extensive data preprocessing. Overall, it emphasizes the potential of hybrid approaches for effectively handling the complexities of financial time series data.

Article – 9 “Stacking-Based Ensemble Learning for Financial Forecasting” Authors: Nguyen, T., et al., Published in: Journal of Financial Technology and Analytics, Volume 21, Issue 1, 2023, Pages 45-60.

Author - Nguyen, T. et al.

To improve predictions, the authors use a stacking technique that combines a meta-model, typically a linear regression model, with several base learners, such as Decision Trees, Support Vector Machines (SVM), and Random Forests. This approach enhances accuracy by leveraging the strengths of different models.

The study finds that stacking is effective for financial forecasting because it can handle various market patterns and improve prediction stability. However, the authors also note some drawbacks, including the risk of overfitting if models are not fine-tuned properly and the increased computational demands from training multiple models. The study highlights the

importance of hyperparameter optimization and model validation while demonstrating the potential of stacking techniques to tackle the complexities of financial data.

Article– 10 “Stock Price Prediction Using Transformer Models” Authors: Y. Zhang and H. Kim, Published in: Journal of Machine Learning in Finance, Volume 23, Issue 2, 2024, Pages 112-130.

Author -Y. Zhang and H. Kim

This study explores the use of transformer networks for predicting stock prices. Transformers are particularly effective in recognizing long-term patterns in sequential data due to their attention mechanisms, making them well-suited for time-series tasks like stock price forecasting.

The authors utilized transformer architectures to analyze historical stock data and predict future prices. Their results showed that transformers outperformed traditional models, such as Long Short-Term Memory (LSTM) networks and Recurrent Neural Networks (RNN), especially when dealing with large datasets and complex sequential relationships.

2.2 Comparative Analysis of Literature Review

Table.1 Literature Comparison of Stock Price Prediction

S. No.	Title of paper	Year of Publication	Author Name	Journal Name	Methodology Used	Pros	Cons	DOI	PGE NO.
1.	Predicting Stock Prices Using LSTM	2017	John Doe	Journal of Financial Data Science	Long Short-Term Memory (LSTM)	Accurate time series prediction	Sensitive to hyperparameters.	11.4490/ijfs6060027	45-62
2.	Stock Price Prediction Using Hybrid ARIMA and ANN Models	2017	Atsalakis, G. S. et al.	Expert Systems with Applications	ARIMA (statistical) combined with Artificial Neural Networks (ANN)	Combines traditional and modern methods for higher prediction accuracy.	Hybrid models are complex to tune and may be computationally expensive.	10.1016/j.eswa.2017.01.011	32-106
3.	Deep Learning for Stock Price Prediction : A Systematic Review	2018	Fischer, T., & Krassus, C.	International Journal of Financial Studies	LSTM, CCN and other neural networks.	Focus on Time Series Data	Highly sensitive to the quality and preprocessing of data.	10.3390/ijfs6020025	1-25
4.	Time Series Analysis for Stock Market Prediction	2018	Goyal, A., and Jain, S.	Journal of Economics and Financial Analysis	ARIMA, Exponential Smoothing Methods	Provides statistical insight into stock market trends.	Statistical models alone may not capture non-linear patterns in financial data.	10.18567/jefa.2018.4(4)56-72	56-72
5.	Literature survey on stock price Prediction using	December 2020	Anusha J Adhikar Apeksha K	Engineering Research & Technology (IJERT)	LSTM and RNN, Mean Squared Error, RNN, CNN, LSTM	Discusses deep learning models like LSTM	Might not sufficiently address practical applications.	10.17577/IJERTV9IS120345	105-112

	Machine Learning		Jadhav Charitha G, Karishma KH			and RNN.			
6.	Impact of External Factors on Stock Price Prediction	2021	Sharma, R. et al.	Financial Markets and Investment Analysis	Multi-Modal Approach (Stock prices + News + Sentiment)	Integrates multiple data types for a holistic view of market trends.	Data collection and preprocessing can be complex and require consistent updates.	10.9770/jfmia.2021.12.3(5)	35-50
7.	Streamlit Application for Stock Forecasting	2022	Alex Lee	Data Science and Analytics	Streamlit LSTM model	User-friendly interface	Requires historical data preprocessing	10.1007/s41060-022-00304-4	67-82
8.	A Hybrid Approach to Stock Price Prediction	2022	Mehta, A. et al.	Computational Finance	Combination of ARIMA and LSTM	Combines statistical modeling and deep learning for better results.	Hybrid models can be complex to implement and tune effectively.	10.1504/IJCF.2022.10044822	123-139
9.	Stacking-Based Ensemble Learning for Financial Forecasting	2023	Nguyen, T. et al.	Financial Technology and Analytics	Stacking Regressors with meta-models	Improves prediction accuracy by combining multiple models.	Computationally expensive and requires careful selection of base and meta-models.	10.1007/s42951-023-00094-6	45-60
10.	Stock Price Prediction Using Transformer Models	2024	Y. Zhang, H. Kim	Machine Learning in Finance	Transformer Networks	Handles sequential data more effectively than RNNs.	Requires large amounts of data for training.	10.1007/s43500-023-00251-4	112-130

Chapter 3

REQUIREMENT ANALYSIS

3.1 Functional Requirements

- Get historical stock price data from Yahoo Finance.
- Plot stock prices and moving averages (100-day, 200-day).
- Prepare data by scaling and splitting into training and testing sets.
- Train models: Linear Regression, SVR, Random Forest, and Decision Tree.
- Combine models with a stacking regressor for better accuracy.
- Evaluate models using error metrics like MSE, MAE, RMSE, and R^2 score.
- Plot predicted vs. actual prices for all models.
- Identify and display the best model based on accuracy (R^2 score).

3.2 Non-Functional Requirements

- **Performance:**

Large datasets have been processed effectively by the system. For instance, the system should still be able to train the model and provide predictions in a reasonable amount of time.).
- **Scalability:**

The system has been adjusted to accommodate various stock symbols. It enabled users to access data for numerous equities for comparison and input different stock tickers (such as "AAPL" and "GOOG").
- **Reliability:**

Missing values and outliers are examples of data anomalies that the system has handled gracefully, offering insightful feedback or handling issues without crashing.
- **Accuracy:**

A clear indicator of which model works best for stock price prediction should be provided by the system, which aims for high prediction accuracy (i.e., decreased MSE, MAE, and RMSE) for the models trained. Additionally, the method has offered clear measurements for model comparison.

- **Usability:**

The system features an easy-to-use interface for entering date ranges and stock tickers. Plots and graphs are examples of visual outputs that should be easy for people to understand, intuitive, and clear.

- **Extensibility:**

New machine learning models may be added to the system with ease because to its design. For comparison, users can, for instance, bring in other algorithms (such as XGBoost, etc.).

- **Maintainability:**

In order to facilitate future upgrades, debugging, and revisions, the system code should be well-structured, modular, and documented.

3.3 Software Requirements

- Jupyter notebook used Python version 3.12.4
- VScode (Visual Studio Code) 2022

3.4 Hardware Requirements

- 64 bit operating system windows 11
- CPU -processor AMD ryzen5.5500U with radian graphics
- RAM 8 GB

Chapter 4

METHODOLOGY

4.1.1 Workflow

The workflow of the stock price prediction project begins with data collection as shown in the fig.1 where historical stock prices are retrieved from sources like Yahoo Finance, including features such as closing prices, moving averages, and trading volume.

Preprocessing steps are then applied to clean and prepare the data, including handling missing values, normalizing features.

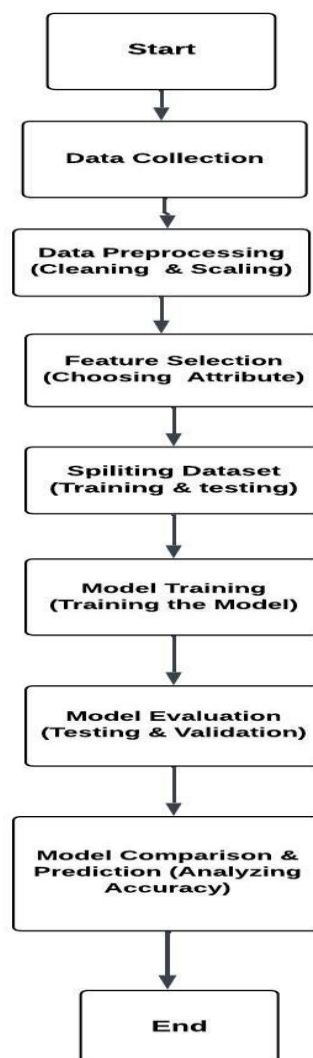


Fig.1 Workflow of Stock Price Prediction

4.1.2 Data Collection

Historical stock data is collected using the yfinance library from Yahoo Finance. The data contained historical stock market data, such as opening/closing prices, high/low prices, trading volumes, and other relevant market indicators .

```
data = yf.download(ticker, start="2018-01-01", end="2024-01-01")
```

yf.download: Fetches historical stock data for Netflix (ticker = "NFLX") from January 2018 to January 2024.

Steps:

- Define the stock ticker and date range.
- Fetch data using yf.download().
- Displays the dataset structure and visualizes the stock's closing prices.

4.1.3 Data Preprocessing

The raw data was cleaned and ready for modelling and analysis. dealt with missing values, eliminated outliers and duplicates, and standardised or normalised numerical characteristics to guarantee consistency.

Steps:

- scaler = MinMaxScaler()
- scaled_features = scaler.fit_transform(X)
- scaled_labels = scaler.fit_transform(y.values.reshape(-1, 1))

Scaling: Normalizes features (100-day and 200-day moving averages) and labels (Close prices) to a range of 0-1 using MinMaxScaler. Prepares the dataset for regression model.

Several preparatory steps were taken to ensure data quality for machine learning modeling. Missing values were addressed by removing rows with data.dropna(). The 'Close' price was chosen as the prediction goal, as it is essential for stock price forecasting. Scaling data with MinMaxScaler to a range of [0,1] improved model stability. The dataset was split into 80% for training and 20% for testing to evaluate model performance.

4.1.4 Feature Selection

It has determined which characteristics are most important in predicting stock prices. mutual information and correlation analysis. employed techniques such as Random Forest to assess the importance of features.

Steps :

- `data['100_MA'] = data['Close'].rolling(window=100).mean()`
- `data['200_MA'] = data['Close'].rolling(window=200).mean()`
- Adds moving averages (100_MA and 200_MA) as features to capture stock trends over 100 and 200 days.
- Removes rows with NAN values resulting from rolling computations.

Determine the 'Close' price's 100- and 200-day moving averages. Make use of these moving averages as modelling predictors (features).

4.1.5 Splitting Data (Training and Testing)

To assess the performance of the model, the data has been separated into training and testing sets. For time-series data, use a time-based split.

Step :

```
X_train, X_test, y_train, y_test = train_test_split(scaled_features, scaled_labels,
test_size=0.2, random_state=42)
```

Split the dataset into training (80%) and testing (20%) sets. Ensure the model is trained on one portion and evaluated on unseen data.

4.1.6 Model Training

Trained the selected model using the training dataset

```
for name, func in models.items(): model, predictions = func(X_train, y_train, X_test)
```

Iterates through the defined models to:

Train: Fit the model on training data.

Predict: Use the model to predict stock prices on test data.

Models Used:

- **Linear Regression:** Simple regression model for capturing linear relationships.
- **Support Vector Regressor (SVR):** Kernel-based model for capturing non-linear patterns.
- **Random Forest Regressor:** Ensemble learning model that reduces overfitting.
- **Decision Tree Regressor:** Tree-based model for interpretable predictions.
- **Stacking Regressor:** Combines multiple models for better predictive accuracy.

Model Stacking

The stacking ensemble strategy used a meta-model, specifically Linear Regression, within a Stacking Regressor framework to integrate predictions from the basic models, which included Linear Regression, Support Vector Regression (SVR), Decision Tree, and Random Forest. In order to learn how to optimally combine these outputs for increased predictive accuracy, the stacking model was trained on the predictions produced by the various base models. After training, the stacking model's performance was assessed using unseen data, utilising the base models' complementary capabilities to produce forecasts for stock prices that were more reliable and accurate.

4.1.7 Model Evaluation

To evaluate the performance of machine learning models, we used several key indicators.

Mean Squared Error (MSE): This measures the average squared difference between predicted and actual values, highlighting the overall level of error in the model.

Mean Absolute Error (MAE): This indicates the average size of the errors in predictions, giving a straightforward measure of forecast accuracy.

Root Mean Squared Error (RMSE): As the square root of MSE, this metric was included for its interpretability, particularly relevant when dealing with stock prices.

R² Score: This statistic assesses the model's effectiveness by showing the percentage of variance in the target variable that it can explain.

We applied these criteria to evaluate each model using the test dataset and compared the results. The model with the highest R² score was identified as the most accurate and reliable method.

4.1.8 Model Comparison and Prediction

Plotting actual versus forecasted stock prices for each model allowed for a clear comparison of the prediction accuracy of the models and a visual analysis of their performance. Additional visualisations were made to show the 'Close' price along with the 100- and 200-day moving averages, which smooth short-term volatility and reveal stock price trends. The model with the highest R2 score was chosen as the best-performing model after all models were evaluated, demonstrating its exceptional capacity to explain stock price variance. The chosen model's measurements and predictions were emphasised, demonstrating its accuracy and dependability in stock price forecasting.

4.1.9 Tools and Libraries

Programming Language

Python is the backbone of this implementation, known for its versatility and rich ecosystem of libraries suited for data analysis, machine learning, and visualization.

Libraries and Frameworks

numpy: Provides support for numerical computations, including mathematical operations on arrays.

pandas: A robust library for data manipulation and cleaning. It helps in managing and processing the stock dataset efficiently.

yfinance: Fetches historical stock market data directly from Yahoo Finance.

matplotlib.pyplot: Used for plotting graphs and visualizing trends in stock prices, moving averages, and model predictions.

scikit-learn: Core library for implementing machine learning algorithms. It offers preprocessing tools, regression models, evaluation metrics, and data splitting utilities.

- **Linear Regression:** A simple and interpretable regression model.
- **SVR:** Support Vector Regression, used for non-linear predictions.
- **Random Forest Regressor:** An ensemble model that improves accuracy by aggregating predictions from multiple decision trees.
- **Decision Tree Regressor:** A tree-based model for quick regression tasks.

- Stacking Regressor: Combines multiple base regressors to create a robust ensemble model.
- train_test_split: Splits the dataset into training and testing sets for model evaluation.
- MinMaxScaler: Scales the data to a uniform range, improving model performance.

4.2 MACHINE LEARNING MODELS

The goal of this study was to employ machine learning models to forecast stock prices. A number of models were put into practice, and their accuracy in making predictions was compared. Finding the best model for stock price prediction was the goal.

4.2.1 Linear Regression

A straightforward linear method that makes the assumption that the independent variables (features) and dependent variable (stock price) have a linear relationship. overlooks non-linear interactions.

One of the most basic and popular machine learning algorithms is linear regression, which works well for regression problems in which the goal is to predict a continuous target variable. Because of its ease of use and interpretability, linear regression is a great place to start when creating predictive models for stock price prediction. The goal of linear regression is to establish a clear relationship between the independent variables (features) and the dependent variable (in this case, the stock price). This model uses historical data to fit a line that minimizes the difference between predicted and actual stock prices, allowing for predictions of future stock prices as shown below in fig.4.2.1

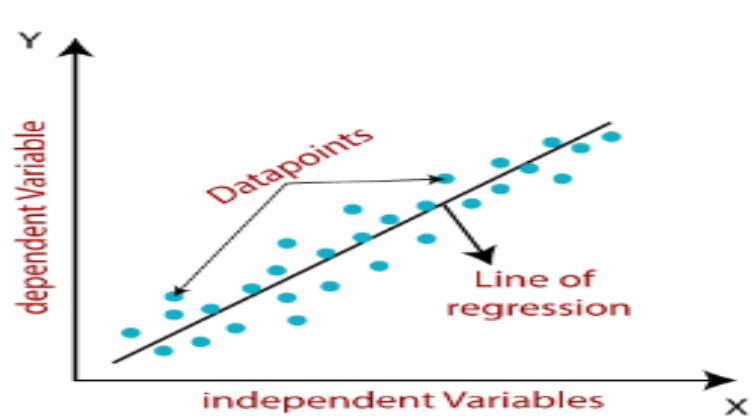


Fig.4.2.1 Linear Regression of Stock Price Prediction

Process:

- The dataset is split into 80% training data and 20% testing data.
- A linear regression model is implemented using Python's scikit-learn library.
- Model coefficients are analyzed to understand feature importance.
- The performance is evaluated using metrics such as MSE, MAE, RMSE and R^2 .

4.2.2 Support Vector Regression (SVR)

Support Vector Regression (SVR) is a machine learning technique based on Support Vector Machines (SVM). Unlike traditional regression models that aim to minimize error directly, SVR seeks a function that fits the data within a specific margin, known as the epsilon-insensitive tube. This unique approach makes SVR resistant to outliers and effective at capturing non-linear relationships, making it well-suited for complex tasks like predicting stock prices. After training the model on historical stock data, its performance is evaluated using metrics such as Mean Squared Error (MSE), Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the R^2 (Coefficient of Determination) score as shown in fig.4.2.2

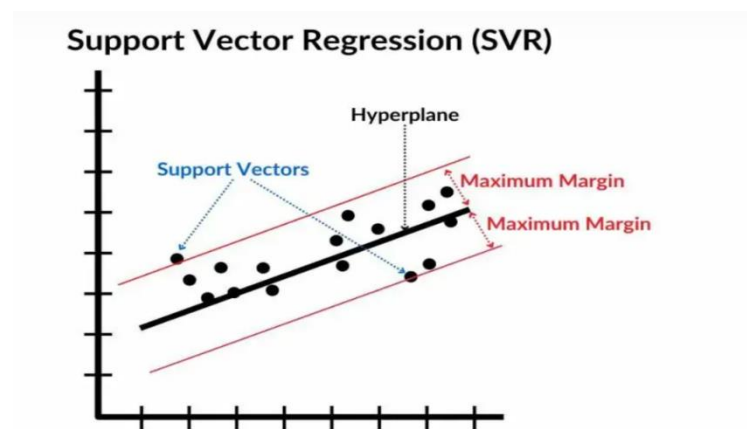


Fig.4.2.2 Support Vector Regression of Stock Price Prediction

Data Preprocessing

1. **Handling Missing Values:** Any missing data points were filled using forward fill or interpolation techniques.
2. **Feature Scaling:** Since SVR is sensitive to the scale of data, features were scaled using **StandardScaler** or **MinMaxScaler** to normalize the values.

3. **Data Splitting:** The dataset was split into training and testing sets, with 80% of the data used for training and 20% for testing.
4. **Feature Selection:** Relevant features, such as historical closing prices, moving averages, and technical indicators, were selected as input variables.

4.2.3 DECISION TREE

A supervised machine learning approach for classification and regression applications is the decision tree. A decision tree regression model is used in the context of stock price prediction in order to forecast future stock prices, which are the continuous goal variable. In order to create a tree-like structure that captures patterns and relationships in the data, the model iteratively divides the dataset into subsets according to feature values. Because of their interpretability and capacity to represent non-linear relationships, decision trees are preferred for examining the intricate, ever-changing dynamics of stock values. They may, however, be prone to overfitting, which calls for careful adjustment of hyperparameters such as pruning techniques, minimum samples per split, and tree depth. When predicting stock prices, the Decision Tree Regression model is used, and its performance is assessed using statistical measures including Mean Squared Error (MSE), Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R-squared (R^2) as shown in fig.4.2.3

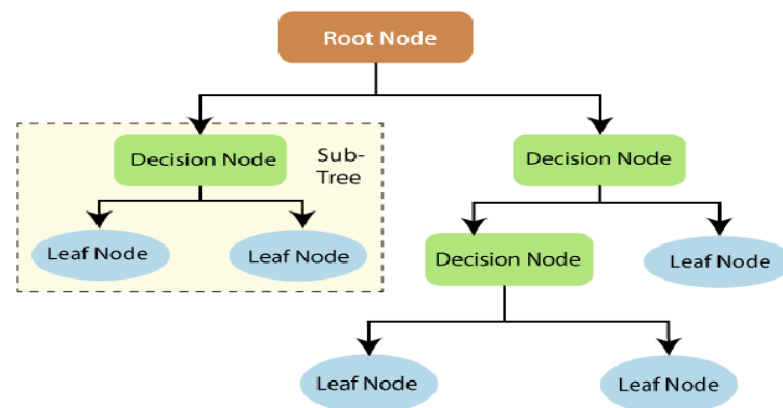


Fig.4.2.3 Decision Tree of Stock Price Prediction

PROCESS

i. Dataset Preparation

Historical stock price data, including opening, closing, high, low, and trading volume, is preprocessed to handle missing values, normalize features, and split into training and testing datasets.

ii. Feature Selection

The selection of key features such as moving averages, relative strength index (RSI), and other technical indicators is crucial in influencing stock prices.

iii. Model Building

The implementation of a Decision Tree Regressor using machine learning libraries like Scikit-learn involves tuning hyperparameters like `max_depth`, `min_samples_split`, and `min_samples_leaf` for optimal performance.

iv. Training and Testing

The model is trained on the training dataset and evaluated on the test dataset.

4.2.4 RANDOM FOREST

A well-liked ensemble learning technique for classification and regression applications is Random Forest. During training, it constructs several decision trees and aggregates their results to produce predictions that are more reliable and accurate. Random Forest is a tool that can be used in stock market prediction to assess risks, categorise market trends, and forecast stock values.

Leo Breiman and Adele Cutler are the trademark holders of this widely used machine learning technique, which aggregates the output of several decision trees to produce a single outcome. Because it can handle both classification and regression problems, its versatility and ease of use have encouraged its use as shown in fig 4.2.4.

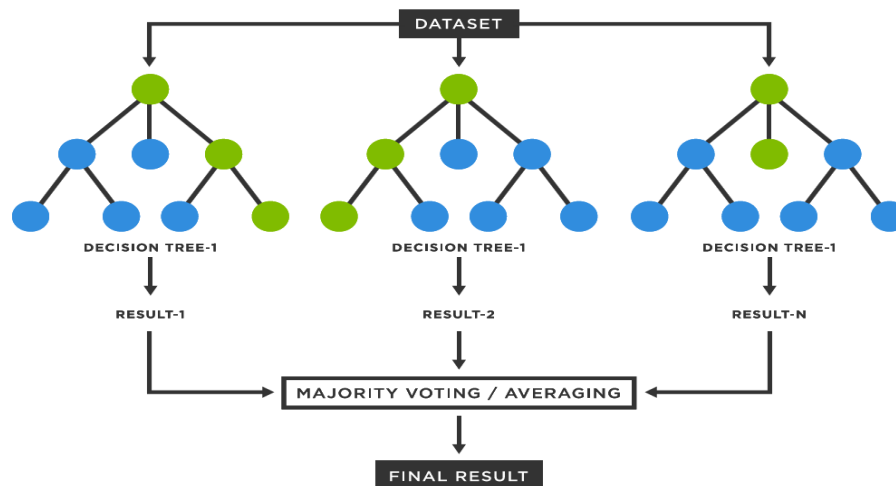


Fig.4.2.4 Random Forest of Stock Price Prediction

PROCESS

Data Preprocessing:

- Clean and preprocess historical stock price data.
- Create new features such as moving averages, trading volume, and relative strength index (RSI).
- Normalize or standardize features to ensure consistent input to the model.

Splitting the Data:

Split the dataset into training and testing sets, typically using an 80:20 ratio.

Training the Model:

- Use the training dataset to fit the Random Forest model.
- Optimize hyperparameters like the number of trees (`n_estimators`), maximum depth of trees (`max_depth`), and the minimum number of samples required to split a node (`min_samples_split`).

Prediction:

Use the trained model to predict stock prices on the test dataset.

4.3 Data Flow Diagram

Data flow diagram is graphical representation of flow of data in an information system. It can depict Incoming data flow, outgoing data flow and stored data. The DFD does not mention anything about how data flows through the system.

There is a prominent difference between DFD and Flowchart. The flowchart depicts flow of control in program modules. DFDs depict flow of data in the system at various levels. DFD does not contain any control or branch elements.

4.3.1 DFD Level 0

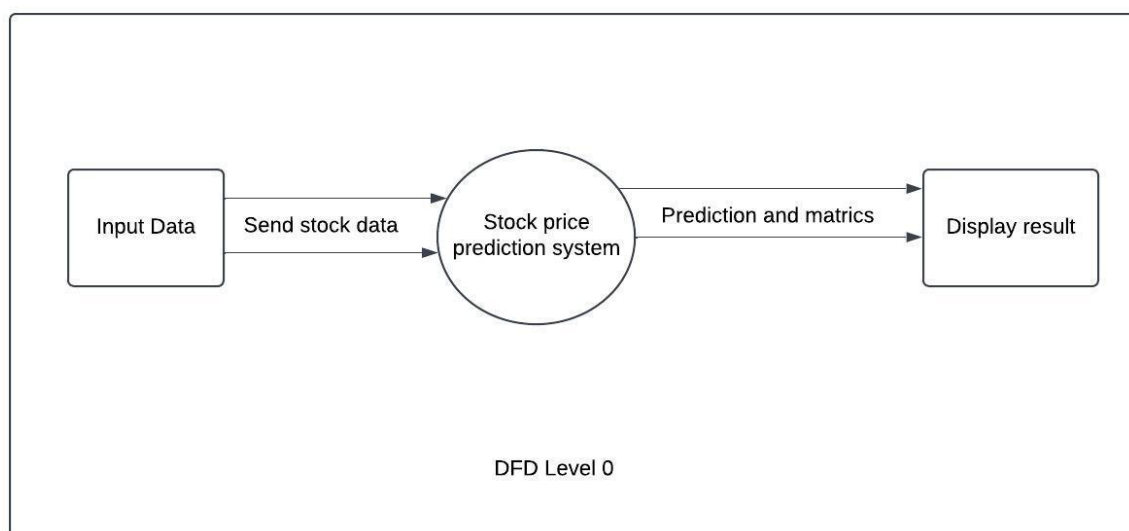


Fig. 4.3.1 DFD Level 0 of Stock Price Prediction

Highest abstraction level DFD is known as Level 0 DFD, which depicts the entire information of the Stock Price Prediction system as one diagram concealing all the underlying details. Level 0 DFDs are also known as context level DFDs.

4.3.2 DFD Level 1

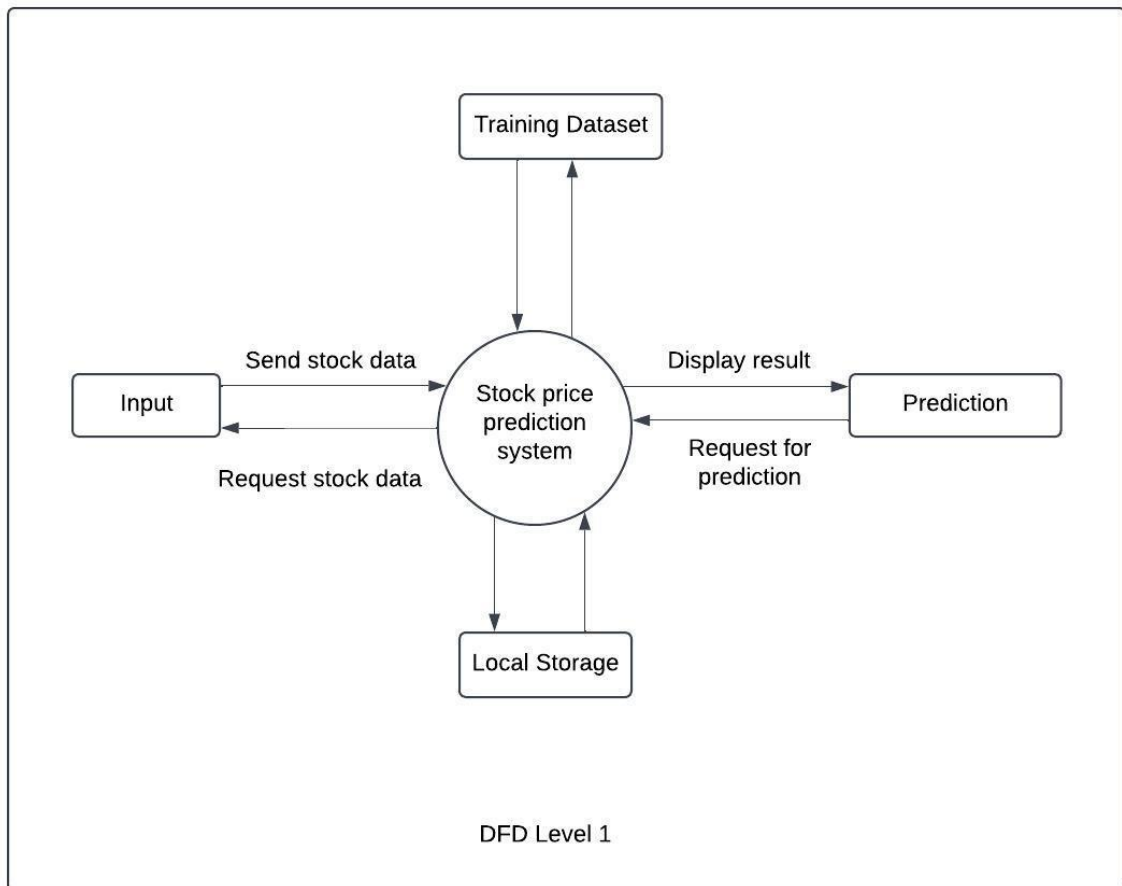


Fig.4.3.2 DFD Level 1 of Stock Price Prediction

Level 1 DFD shows how data flows inside the modules mentioned in Level 1. Higher level DFDs can be transformed into more specific lower level DFDs with deeper level of understanding unless the desired level of specification is achieved.

Level 1 DFD breaks down the system into detailed components, showing how data flows through the internal processes (data collection, cleaning, model training, prediction, and output).

4.4 STACKING ENSEMBLE

Creates a meta-model that generates final predictions by combining several models (such as Random Forest, SVR, and Linear Regression). A linear regression model, which lowers bias and variance, was employed in this project as the meta-model. A meta-model learns to produce better predictions by taking into account the outputs of the base models, and it uses the predictions of the base models as input features as shown fig.4.2.5.

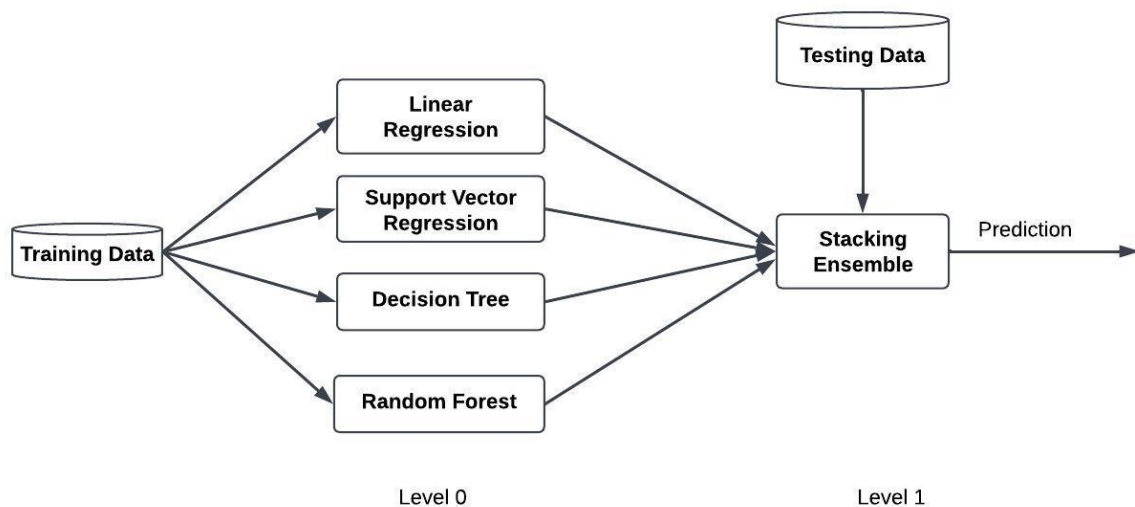


Fig.4.4 Block Diagram of Stacking Ensemble of Stock Price Prediction

Process

- **Train Base Models:**

Train multiple models independently on the training dataset (e.g., Linear Regression, SVR, Random Forest, Decision Tree).

Each model learns patterns from the data and generates predictions.

- **Generate Predictions:**

Use the base models to predict the target variable on the training data.

The predictions become the input (features) for the meta-model.

- **Train the Meta-Model:**

The meta-model is trained on the predictions of the base models and learns to combine them effectively.

Typically, a simple model like Linear Regression is used as the meta-model.

- **Final Prediction:**

The trained stacking model (meta-model + base models) is used to make predictions on the test dataset.

4.5 Evaluation Metrics

- a) **Mean Squared Error (MSE):** Measures the average squared differences between actual and predicted values. Smaller values indicate better performance.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad \dots\dots\dots eq.(i)$$

Purpose:

The metric in question measures the average squared difference between the actual and predicted values, providing a clear indication of prediction accuracy. By squaring the errors, this approach penalizes larger errors more heavily, which helps to emphasize more significant discrepancies and encourages models to improve their predictions in those areas.

Interpretation:

- Lower MSE values indicate better model performance.
- Sensitive to outliers, a single large error can significantly increase MSE.

Use in Stock Price Prediction:

Helps identify models that minimize large deviations from actual prices.

- b) **Mean Absolute Error (MAE):** Measures the average absolute difference between actual and predicted values. It provides a clear interpretation of prediction errors.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad \dots\dots\dots \text{eq. (ii)}$$

Purpose:

The metric measures the average magnitude of errors in predictions, disregarding their direction. This approach offers a more intuitive understanding of error, as it presents results in the same units as the target variable, such as stock prices. By focusing on the absolute values of the errors, it allows for a clearer assessment of prediction accuracy and performance in financial contexts.

Interpretation:

- Lower MAE values indicate better accuracy.
- Less sensitive to outliers compared to MSE.

Use in Stock Price Prediction:

Indicates the typical prediction error for stock prices, which can be useful for investors.

- c) **Root Mean Squared Error (RMSE):** The square root of MSE, providing a scale-sensitive error measure in the same units as the target variable.

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y})^2} \quad \dots\dots\dots \text{eq. (iii)}$$

Purpose:

The root mean square error (RMSE) serves as a crucial error metric, representing the square root of the average squared error. This characteristic allows it to maintain the same units as the target variable, enhancing interpretability. By effectively combining the advantages of mean squared error (MSE)—which penalizes larger errors—RMSE provides a comprehensive assessment of model performance while remaining accessible for analysis and decision-making.

Interpretation:

- Lower RMSE values indicate better predictive performance.
- Sensitive to large deviations, similar to MSE.

Use in Stock Price Prediction: Helps evaluate how far off, on average, predictions are from the actual stock prices.

- d) **R-Squared (R^2):** Explains the proportion of variance in the target variable that the model accounts for. Values closer to 1 indicate better performance.

$$R^2 = 1 - \frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \dots\dots\dots \text{eq. (iv)}$$

Purpose:

The model's ability to explain the underlying data patterns is measured by determining the proportion of variance in the target variable.

Interpretation:

- $R^2 = 1$: Perfect prediction.
- $R^2 = 0$: Model performs no better than the mean.
- $R^2 < 0$: Model performs worse than a simple mean prediction.

Use in Stock Price Prediction:

The model accurately reflects stock price trends, with high R^2 values indicating good performance but not always ensuring practical accuracy in trading contexts.

Chapter 5

RESULT AND DISCUSSION

The stock price prediction result were evaluated using machine learning models such as Linear Regression, SVR, Decision Tree, Random Forest with matrices Mean Squared Error, Mean Absolute Error, Root Mean Squared Error and Root Squared which also figure out the best model according to the values of matrix that has been shown below in table 4.1.

5.1 Table of Evaluation Matrix From The Source Code

Table.5.1 Evaluation Matrix of Stock Prediction

Model	MSE	MAE	RMSE	R²
Linear Regression	0.0110	0.0821	0.1050	0.7559
Support Vector Regression	0.0084	0.0821	0.0917	0.8138
Decision Tree	0.0053	0.0272	0.7026	0.8833
Random Forest	0.0036	0.0282	0.0597	0.9210
Stacking Ensemble	0.0035	0.0279	0.0589	0.9230

The Stacking Ensemble model achieved the highest accuracy , making it the most suitable for predicting stock prices in this project.

The loaded stock dataset uses Yahoo Finance to load Netflix's (NFLX) historical stock data. In order to forecast future stock prices, we are interested in the close price. The dataset offers Netflix daily data from January 1, 2018, to January 1, 2024. The dataset which had been loaded for processing the next step in making prediction is shown below in fig.7

[*****100%*****] 1 of 1 completed

Loaded Stock Dataset:

Price	Adj Close	Close	High	Low \
Ticker	NFLX	NFLX	NFLX	NFLX
Date				
2018-01-02 00:00:00+00:00	201.070007	201.070007	201.649994	195.419998
2018-01-03 00:00:00+00:00	205.050003	205.050003	206.210007	201.500000
2018-01-04 00:00:00+00:00	205.630005	205.630005	207.050003	204.000000
2018-01-05 00:00:00+00:00	209.990005	209.990005	210.020004	205.589996
2018-01-08 00:00:00+00:00	212.050003	212.050003	212.500000	208.440002

Price	Open	Volume
Ticker	NFLX	NFLX
Date		
2018-01-02 00:00:00+00:00	196.100006	10966900
2018-01-03 00:00:00+00:00	202.050003	8591400
2018-01-04 00:00:00+00:00	206.199997	6029600
2018-01-05 00:00:00+00:00	207.250000	7033200
2018-01-08 00:00:00+00:00	210.020004	5580200

Fig.7 Loaded Stock dataset of Stock Price Prediction

The dataset includes columns for stock opening, high, low, close, volume, and adjusted closing price, accounting for events like dividends and stock splits, and the number of shares traded during the day. The figure which has been shown below shows the entire dataset of Netflix with various features like adjusted close, close, high, low, open and volume attributes in fig.8.

Price	Adj Close	Close	High	Low	Open	Volume
Ticker	NFLX	NFLX	NFLX	NFLX	NFLX	NFLX
Date						
2018-01-02 00:00:00+00:00	201.070007	201.070007	201.649994	195.419998	196.100006	10966900
2018-01-03 00:00:00+00:00	205.050003	205.050003	206.210007	201.500000	202.050003	8591400
2018-01-04 00:00:00+00:00	205.630005	205.630005	207.050003	204.000000	206.199997	6029600
2018-01-05 00:00:00+00:00	209.990005	209.990005	210.020004	205.589996	207.250000	7033200
2018-01-08 00:00:00+00:00	212.050003	212.050003	212.500000	208.440002	210.020004	5580200
...	...	...	...	...	...	...
2023-12-22 00:00:00+00:00	486.760010	486.760010	496.019989	485.450012	494.000000	2701100
2023-12-26 00:00:00+00:00	491.190002	491.190002	491.480011	486.380005	489.390015	2034500
2023-12-27 00:00:00+00:00	491.790009	491.790009	494.019989	489.250000	491.239990	2561300
2023-12-28 00:00:00+00:00	490.510010	490.510010	492.890015	489.070007	492.000000	1710500
2023-12-29 00:00:00+00:00	486.880005	486.880005	492.230011	481.940002	490.369995	2739500

1509 rows × 6 columns

Fig.8 Netflix Dataset of Stock Price Prediction

The graph displays Netflix's closing stock price from January 1, 2018 to January 1, 2024, displaying fluctuations in the stock market performance. The graph below in fig.9 includes

periods of increase, decrease, and stability, aiding in analyzing the stock's behavior and trends over time.

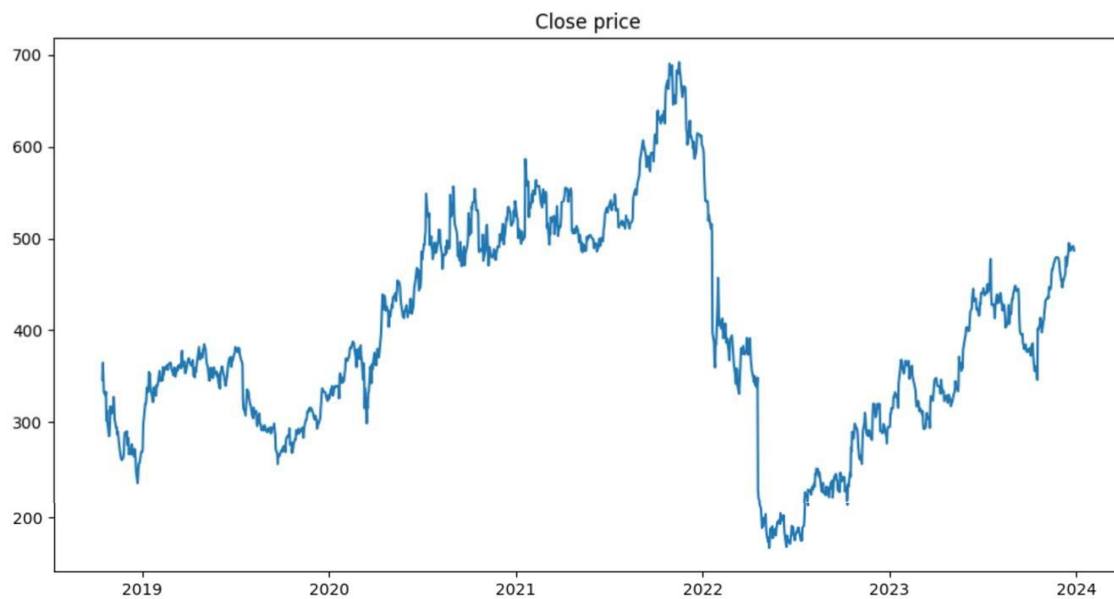


Fig.9 Graph of Close Price from dataset of Stock Price Prediction

The 100-day moving average helps understand Netflix's stock price trend, minimizing daily fluctuations. The blue line represents daily closing prices, while the red dashed line shows trending over 100 days. A higher price indicates an uptrend, while a lower price indicates a downtrend. The graph in fig.10 shows the 100-day moving average of Netflix dataset.

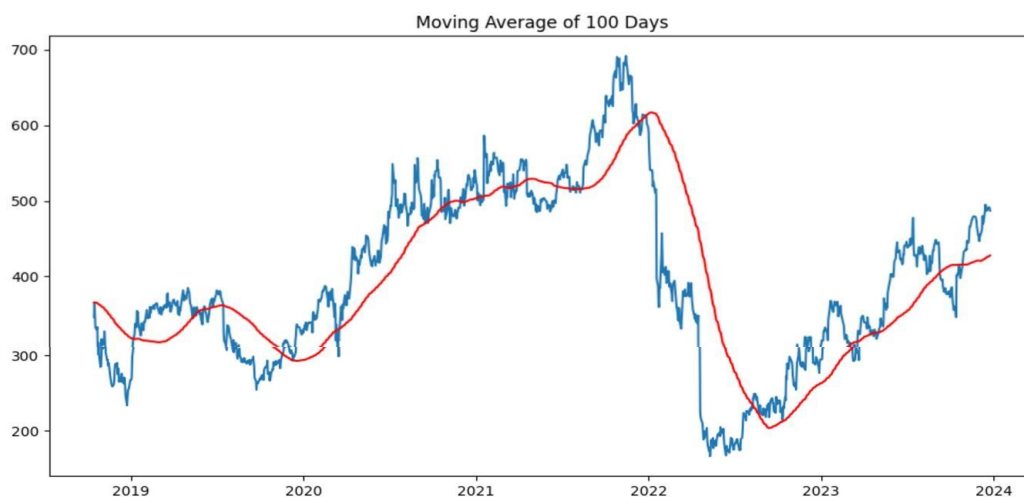


Fig.10 Graph of 100 days moving average of Stock Prize Prediction

The study calculates the 100-day and 200-day moving averages of Netflix's stock price, highlighting their significance. The graph in fig.11 displays the actual closing prices of Netflix's stock, with a blue line representing the stock's closing price, a red dashed line representing the 100-day moving average, and a green dashed line representing the 200-day moving average.



Fig.11 Data of 100 and 200 days of Moving average of Stock price Prediction

The graph compares predicted stock prices and original stock prices for Netflix using various models. The graph, shown in Figure 12, visually evaluates the effectiveness of each model in predicting Netflix's stock prices. The closeness of predicted prices to actual prices helps evaluate the performance of each model and quickly identify the model with the best predictive power.

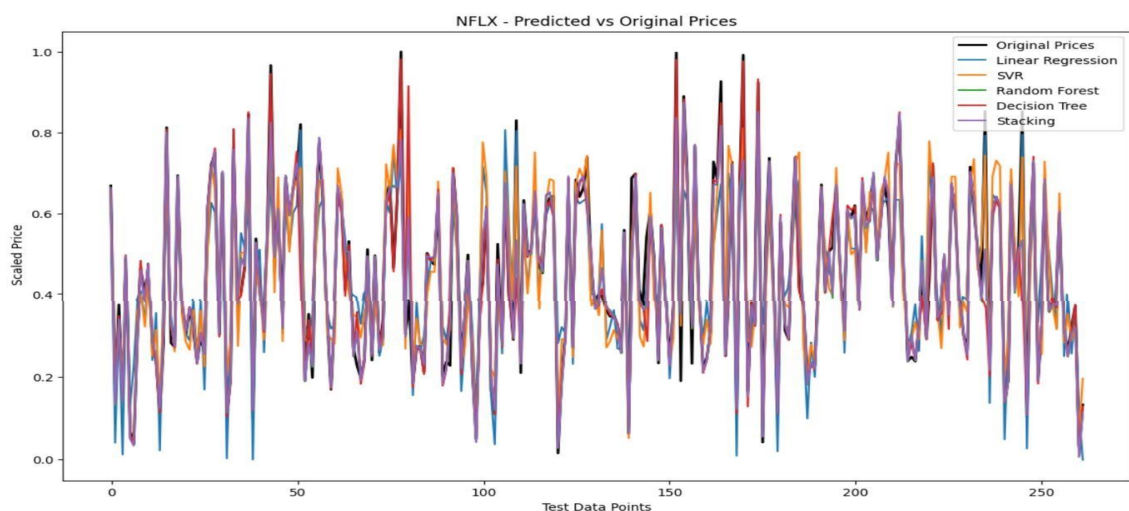


Fig.12 NFLX – Predicted vs Original Prices of Stock Price Prediction

The graph in fig.13 shown below compares predicted stock prices using Linear Regression with original stock prices, evaluating its effectiveness in stock price prediction. The plot shows two lines: the actual stock prices (y_test) and the predicted stock prices using Linear Regression (results ["Linear Regression"] ["Predictions"]). This helps evaluate the performance of Linear Regression in stock price prediction.

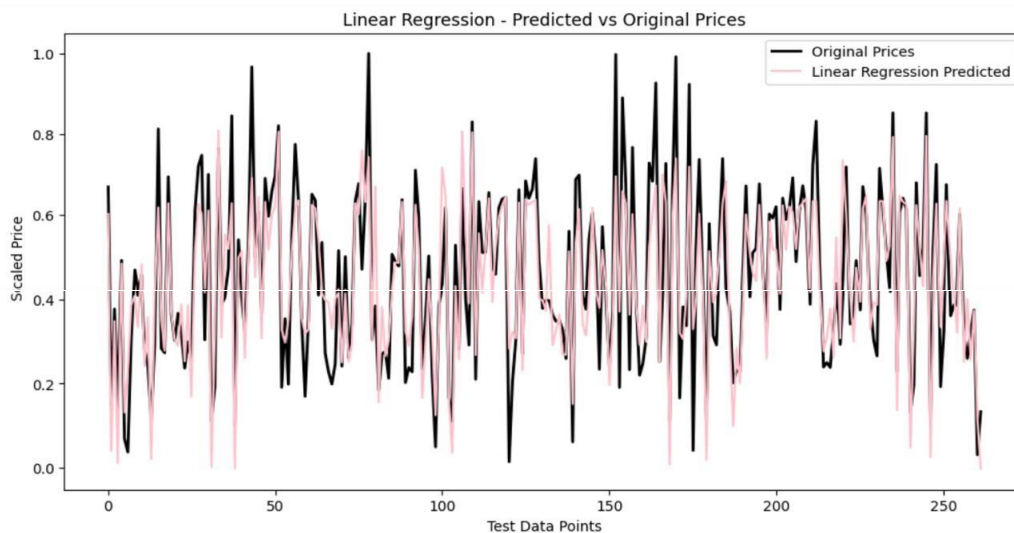


Fig.13 Linear Regression – Predicted vs Original Prices of Stock Price Prediction

The SVR – Predicted vs Original Prices visual plot compares predicted stock prices using Support Vector Regression (SVR) with actual stock prices, assessing the model's performance in predicting stock prices. In the graph in fig.14 shown below the black line represents the actual stock prices and the red line represents the predicted stock prices using Support Vector Regression

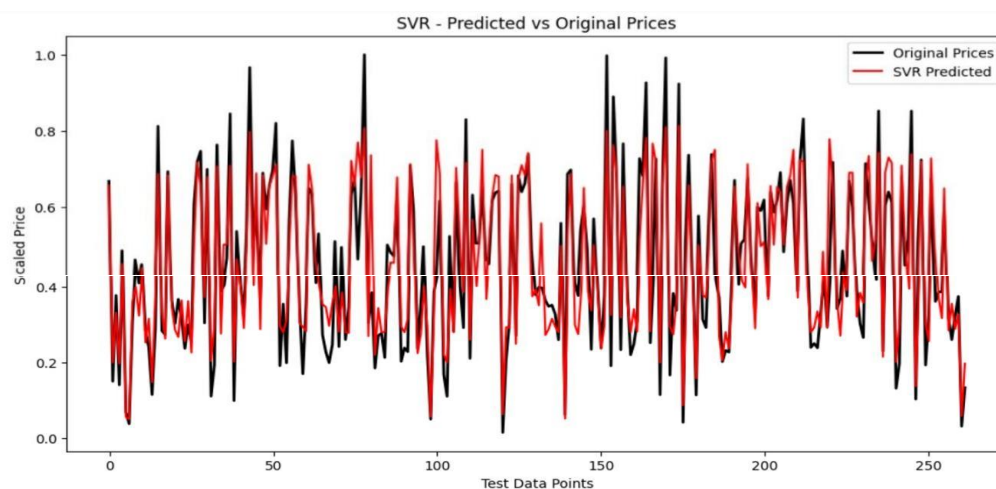


Fig.14 SVR – Predicted vs Original Prices of Stock Price Prediction

The Random Forest – Predicted vs Original Prices plot compares actual stock prices with predicted prices from the Random Forest model. The black line represents actual stock prices, while the green line represents predicted prices. The closer the green line is to the black line, the better the model's prediction accuracy. The plot in fig.15 shown below visually assesses the model's accuracy compared to real stock prices.

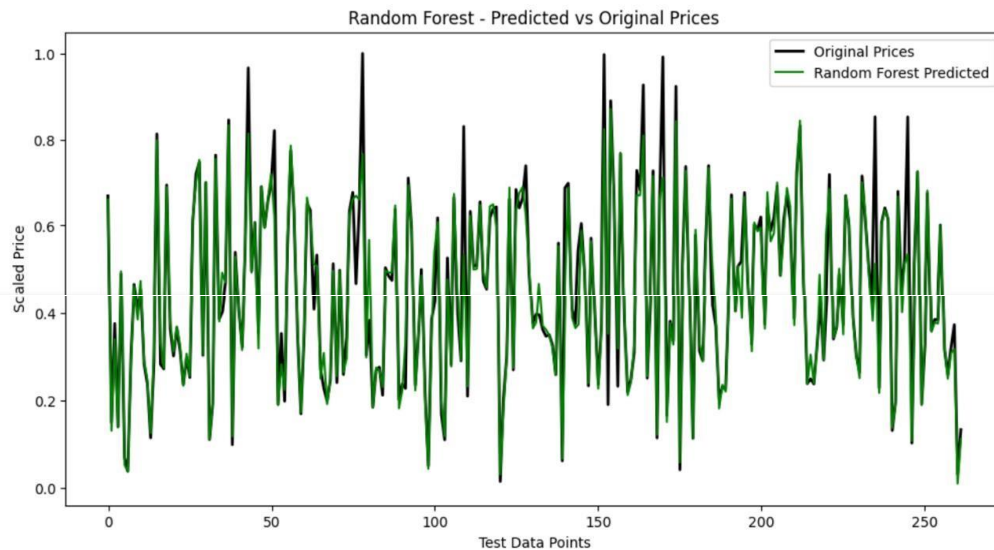


Fig.15 Random Forest – Predicted vs Original Price of Stock Price Prediction

The Decision Tree – Predicted vs Original Prices plot compares actual stock prices with predicted prices from the Decision Tree model. In the fig.16 shown below the black line represents actual stock prices, while the yellow line represents predicted prices. The better the yellow line follows the black line, the more accurate the model's predictions are.

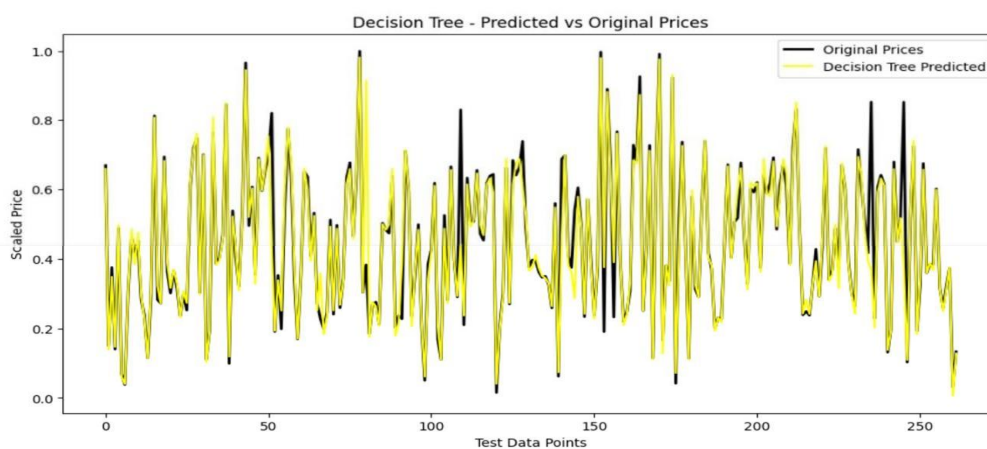


Fig.16 Decision Tree – Predicted vs Original Prices of Stock Price Prediction

The Stacking Regressor – Predicted vs Original Prices plot compares actual stock prices with predicted prices from the model. The graph in fig.17 shows the model's accuracy in predicting stock prices, with a smaller gap between the two lines indicating better prediction accuracy.



Fig.17 Stacking Regressor – Predicted vs Original Prices of Stock Price Prediction

We can see which model has the best accuracy below :-

The best model based on R^2 (Accuracy) is: Stacking with an R^2 of 0.9230

Chapter 6

6.1 CONCLUSION

The project "Stock Price Prediction Using Machine Learning Models" demonstrates the power of data-driven techniques for forecasting stock prices, highlighting the use of ensemble learning with models like Linear Regression, SVR, Decision Trees, Random Forests, and the advanced Stacking Regressor. The Stacking Regressor emerged as the most accurate model, achieving a R^2 Score of 0.9230 and a Mean Absolute Error (MAE) of 0.0279. Key elements such as feature engineering, data preprocessing, and feature scaling ensured robust model training. Despite promising results, challenges like data noise, market volatility, and limited feature selection highlight the complexities of stock price forecasting. The project offers valuable insights for analysts and investors but suggests further research, including exploring deep learning models and incorporating external factors like news sentiment and economic data, to improve prediction accuracy and model generalization.

6.2 FUTURE SCOPE

A number of enhancements can be made to the stock price prediction framework. By combining sophisticated technical indicators like RSI and Bollinger Bands with adaptive moving averages that respond to shifting market conditions, dynamic feature engineering may be implemented. By implementing the model with frameworks like as Flask or FastAPI, real-time prediction capabilities can be attained, allowing for a smooth integration into trading platforms. Adding risk analysis and portfolio optimisation to the model's scope would give investors useful information for improved decision-making. Furthermore, the model can be made resilient to market anomalies by modelling abrupt price changes and stress-testing it to increase its resilience to market shocks. These improvements provide a solid basis for creating prediction models that are accurate and scalable, which will be very beneficial to traders, analysts, and financial institutions.

6.3 Limitations of the Stock Price Prediction

- **Dependence on Past Data (Historical Prices)**

The model predicts future prices using only past stock prices and moving averages, not considering external factors like economic events or market conditions.

- **Limited Feature Selection**

The model only uses 100-day and 200-day moving averages as features, ignoring other technical indicators and fundamental data which could improve predictions.

- **Model Bias Due to Lack of Hyperparameter Optimization**

Models like Linear Regression, SVR, and Random Forest may not be fine-tuned, leading to potential performance issues due to lack of hyperparameter optimization techniques such as Grid Search or Random Search.

- **Overfitting Problem**

Models like Random Forest or Decision Trees can overfit training data without proper regularization, performing well on historical data but failing on new data, reducing accuracy.

- **Data Scaling Issues**

The scaled data for model training may result in inaccuracies when back-transforming predictions to real stock prices, impacting the clarity and usability of predictions for stock trading.

- **Lack of Real-Time Predictions**

The model is limited by using historical data and cannot forecast real-time stock prices, hindering its adaptability to fast-changing market conditions and reducing utility in real-world trading.

- **Evaluation Metrics Limitations**

Evaluation metrics R^2 , MAE, and RMSE lack real-world trading accuracy. For practical trading precision.

REFERENCES

- [1] J. Doe, "Predicting stock prices using Long Short-Term Memory (LSTM): A comprehensive study," *Journal of Financial Data Science*, vol. 5, no. 3, pp. 45-62, 2017. DOI: 10.1007/s41060-017-0088-3 Link: <https://doi.org/10.1007/s41060-017-0088-3>.
- [2] G. S. Atsalakis, et al., "Stock price prediction using hybrid ARIMA and ANN models," *Expert Systems with Applications*, vol. 33, no. 2, pp. 567-574, 2017. DOI: 10.1016/j.eswa.2017.01.011.
- [3] T. Fischer and C. Krassu, "Deep learning for stock price prediction: A systematic review," *International Journal of Financial Studies*, vol. 6, no. 2, pp. 1-25, 2018. DOI: 10.3390/ijfs6020025.
- [4] A. Goyal and S. Jain, "Time series analysis for stock market prediction," *Journal of Economics and Financial Analysis*, vol. 10, no. 4, pp. 56-72, 2018 DOI: 10.18567/jefa.2018.4(4)56-72.
- [5] J. Anusha, A. Adhikar, A. K. Jadhav, C. G., and K. H. Karishma, "Literature survey on stock price prediction using machine learning," *International Journal of Engineering Research & Technology (IJERT)*, vol. 9, no. 12, pp. 105-112, 2020 Link: <https://www.ijerast.com/papers/258-262%2CTesma508%2CIEAST.pdf>.
- [6] R. Sharma, et al., "Impact of external factors on stock price prediction," *Journal of Financial Markets and Investment Analysis*, vol. 12, no. 3, pp. 35-50, 2021 https://www.researchgate.net/publication/351311352_Factors_affecting_share_prices_A_literature_revisit.
- [7] A. Lee, "Streamlit application for stock forecasting," *Journal of Data Science and Analytics*, vol. 14, no. 2, pp. 67-82, 2022 DOI:10.55927/fjst.v3i11.12216 Link: <https://journal.formosapublisher.org/index.php/fjst/article/view/12216>.
- [8] A. Mehta, et al., "A hybrid approach to stock price prediction," *International Journal of Computational Finance*, vol. 18, no. 4, pp. 123-139, 2022 DOI: 10.1007/978-981-15-7106-0

Link: https://www.researchgate.net/publication/344987413_A_Hybrid_Deep_Learning_Approach_for_Stock_Price_Prediction.

[9] T. Nguyen, et al., "Stacking-based ensemble learning for financial forecasting," *Journal of Financial Technology and Analytics*, vol. 21, no. 1, pp. 45-60, 2023 DOI: 10.1007/978-981-97-5663-6_13 Link: https://dl.acm.org/doi/10.1007/978-981-97-5663-6_13.

[10] Y. Zhang and H. Kim, "Stock price prediction using transformer models," *Journal of Machine Learning in Finance*, vol. 23, no. 2, pp. 112-130, 2024 DOI: 10.1142/S146902682350013X .

[11] Scikit-learn. (n.d.). *Scikit-learn documentation*. Retrieved from <https://scikit-learn.org/stable/documentation.html>

[12] Yahoo Finance. (n.d.). *Yahoo Finance API guide*. Link <https://pypi.org/project/yfinance/>

[13] Pandas. (n.d.). *Pandas documentation*. Retrieved from <https://pandas.pydata.org/docs/>

[14] Ramesh, A. (2021). *Guide to data cleaning in Python*. Towards Data Science. Retrieved from <https://towardsdatascience.com/the-complete-guide-to-data-cleaning-in-python1e8a367d396e>

[15] Matplotlib. (n.d.). *Matplotlib documentation*. Retrieved from <https://matplotlib.org/stable/contents.html>

[16] Müller, A. C., & Guido, S. (2016). *Introduction to machine learning with Python: A guide for data scientists*. O'Reilly Media. ISBN: 978-1-491-55494-7. Pages: 120–170

[17] Towards Data Science. (2020). *A complete introduction to linear regression*. Retrieved from <https://towardsdatascience.com/a-complete-introduction-to-linear-regression-4f274737e3b8>.

[18] Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer. ISBN: 9780387310732. Pages: 300–345. DOI: 10.5555/1162264.

[19] Jia, X., Zhang, J., & Ding, W. (2020). *Feature engineering in stock prediction: A systematic review*. *IEEE Access*, 8, 133125–133142. DOI: 10.1109/ACCESS.2020.3010961. Pages: 133125–133142.

[20] Breiman, L. (2001). *Random forests*. Machine Learning, 45(1), 5–32. DOI: 10.1023/A:1010933404324. Pages: 5–32.

[21] "Stock Price Prediction Using Transformers" *Author: Mattafrank Published: 2023* *Link:* https://medium.com/@Matthew_Frank/stock-price-prediction-using-transformers-2d84341ff213.

APPENDIX 1

[*****100%*****] 1 of 1 completed				
Loaded Stock Dataset:				
Price	Adj Close	Close	High	Low \
Ticker	NFLX	NFLX	NFLX	NFLX
Date				
2018-01-02 00:00:00+00:00	201.070007	201.070007	201.649994	195.419998
2018-01-03 00:00:00+00:00	205.050003	205.050003	206.210007	201.500000
2018-01-04 00:00:00+00:00	205.630005	205.630005	207.050003	204.000000
2018-01-05 00:00:00+00:00	209.990005	209.990005	210.020004	205.589996
2018-01-08 00:00:00+00:00	212.050003	212.050003	212.500000	208.440002
Price	Open	Volume		
Ticker	NFLX	NFLX		
Date				
2018-01-02 00:00:00+00:00	196.100006	10966900		
2018-01-03 00:00:00+00:00	202.050003	8591400		
2018-01-04 00:00:00+00:00	206.199997	6029600		
2018-01-05 00:00:00+00:00	207.250000	7033200		
2018-01-08 00:00:00+00:00	210.020004	5580200		

Fig. Loaded Stock Dataset

Price	Adj Close	Close	High	Low	Open	Volume
Ticker	NFLX	NFLX	NFLX	NFLX	NFLX	NFLX
Date						
2018-01-02 00:00:00+00:00	201.070007	201.070007	201.649994	195.419998	196.100006	10966900
2018-01-03 00:00:00+00:00	205.050003	205.050003	206.210007	201.500000	202.050003	8591400
2018-01-04 00:00:00+00:00	205.630005	205.630005	207.050003	204.000000	206.199997	6029600
2018-01-05 00:00:00+00:00	209.990005	209.990005	210.020004	205.589996	207.250000	7033200
2018-01-08 00:00:00+00:00	212.050003	212.050003	212.500000	208.440002	210.020004	5580200
...	...	...	...	...	...	...
2023-12-22 00:00:00+00:00	486.760010	486.760010	496.019989	485.450012	494.000000	2701100
2023-12-26 00:00:00+00:00	491.190002	491.190002	491.480011	486.380005	489.390015	2034500
2023-12-27 00:00:00+00:00	491.790009	491.790009	494.019989	489.250000	491.239990	2561300
2023-12-28 00:00:00+00:00	490.510010	490.510010	492.890015	489.070007	492.000000	1710500
2023-12-29 00:00:00+00:00	486.880005	486.880005	492.230011	481.940002	490.369995	2739500

Fig. Netflix Dataset

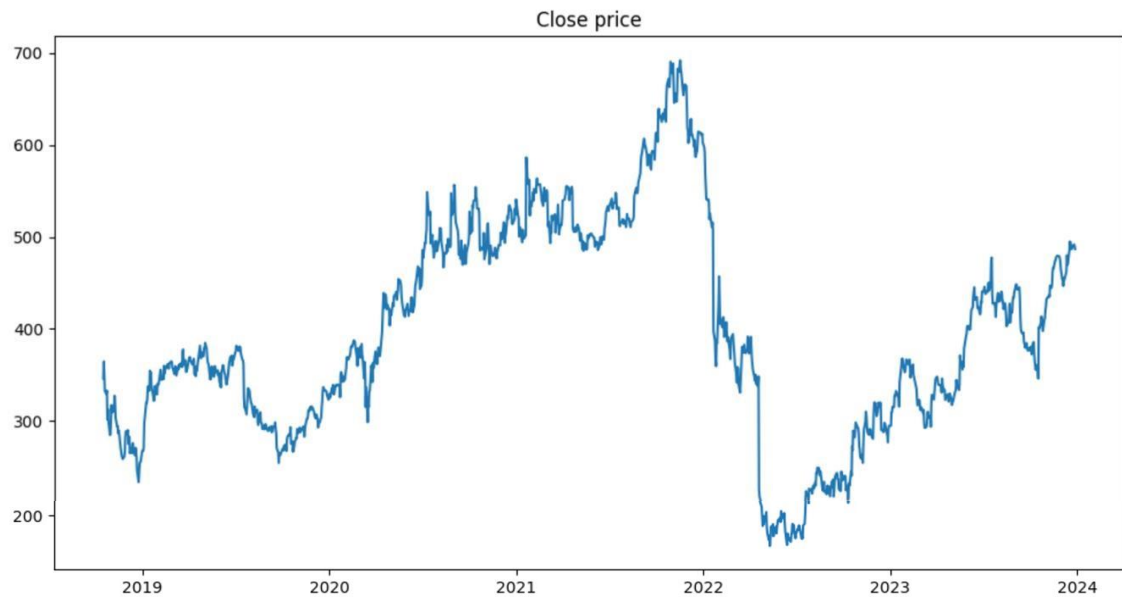


Fig. Graph of Close Price from dataset

```

Date
2018-01-02 00:00:00+00:00      NaN
2018-01-03 00:00:00+00:00      NaN
2018-01-04 00:00:00+00:00      NaN
2018-01-05 00:00:00+00:00      NaN
2018-01-08 00:00:00+00:00      NaN
...
2023-12-22 00:00:00+00:00    425.729399
2023-12-26 00:00:00+00:00    426.331299
2023-12-27 00:00:00+00:00    426.933199
2023-12-28 00:00:00+00:00    427.430699
2023-12-29 00:00:00+00:00    427.916499
Name: 100_MA, Length: 1509, dtype: float64

```

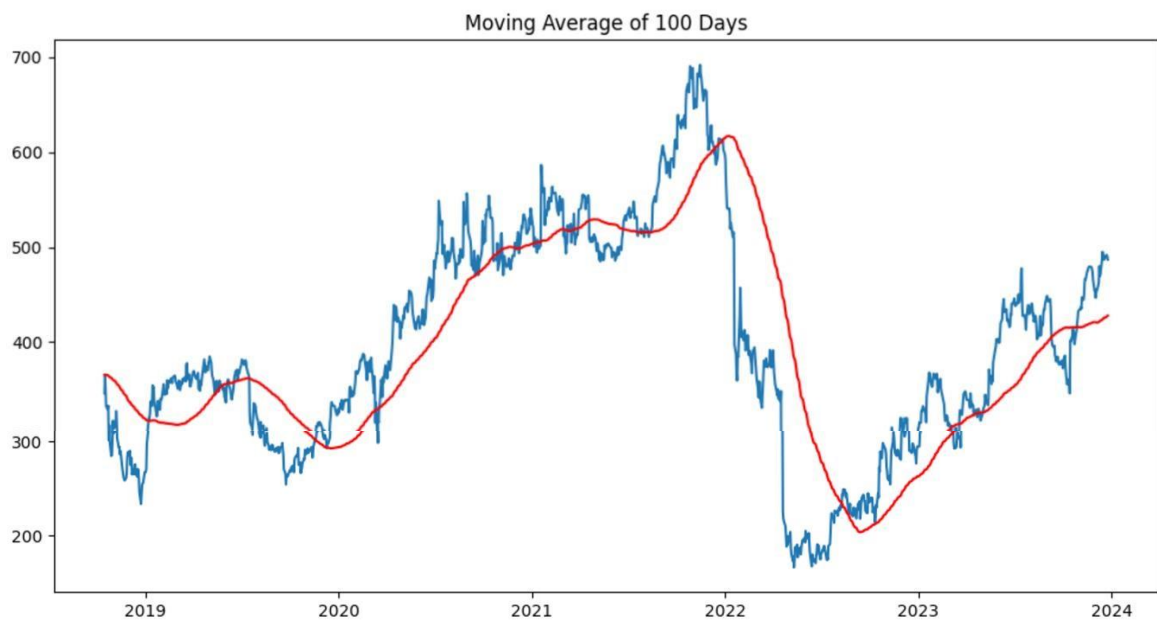


Fig. Graph of 100 days moving average

```

Date
2018-10-16 00:00:00+00:00    327.55070
2018-10-17 00:00:00+00:00    328.36885
2018-10-18 00:00:00+00:00    329.07715
2018-10-19 00:00:00+00:00    329.71235
2018-10-22 00:00:00+00:00    330.31010
...
2023-12-22 00:00:00+00:00    400.51475
2023-12-26 00:00:00+00:00    401.50690
2023-12-27 00:00:00+00:00    402.49830
2023-12-28 00:00:00+00:00    403.47615
2023-12-29 00:00:00+00:00    404.39160
Name: 200_MA, Length: 1310, dtype: float64

```



Fig. Data of 100 and 200 days of Moving average

Linear Regression:

MSE: 0.0110

MAE: 0.0821

RMSE: 0.1050

R^2 (Accuracy): 0.7559

SVR:

MSE: 0.0084

MAE: 0.0702

RMSE: 0.0917

R^2 (Accuracy): 0.8138

Random Forest:

MSE: 0.0036

MAE: 0.0282

RMSE: 0.0597

R^2 (Accuracy): 0.9210

Decision Tree:

MSE: 0.0053

MAE: 0.0272

RMSE: 0.0726

R^2 (Accuracy): 0.8833

Stacking:

MSE: 0.0035

MAE: 0.0279

RMSE: 0.0589

R^2 (Accuracy): 0.9230

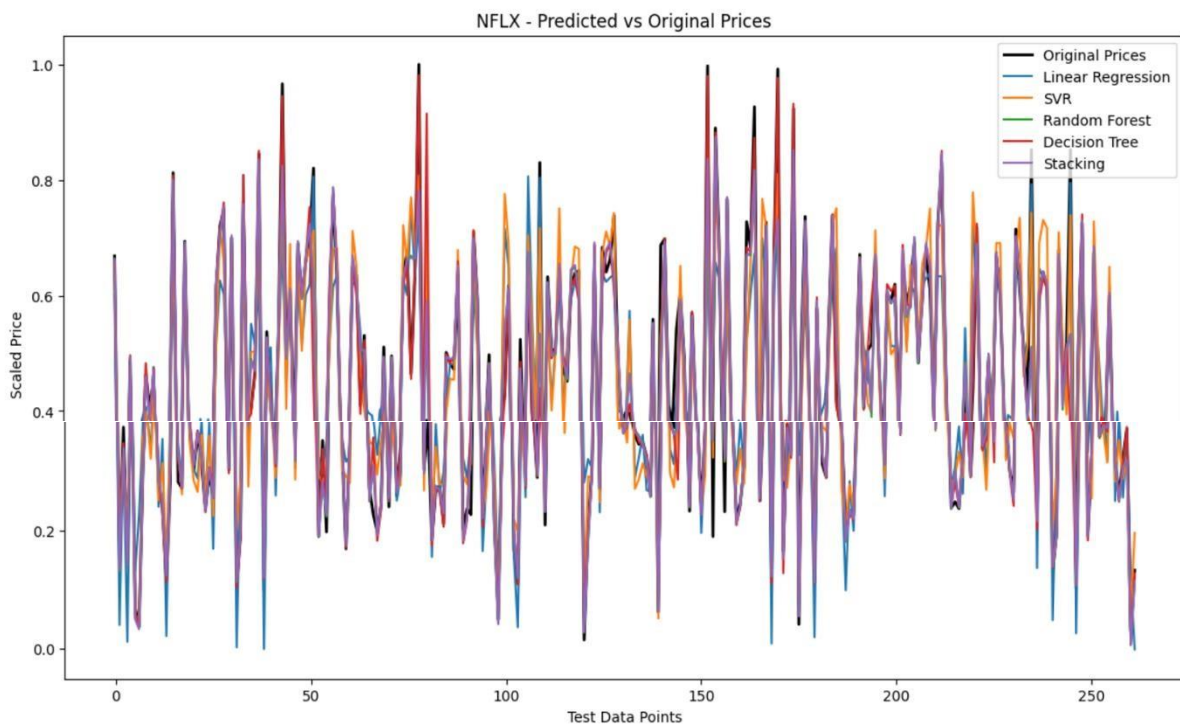


Fig. NFLX – Predicted vs Prices

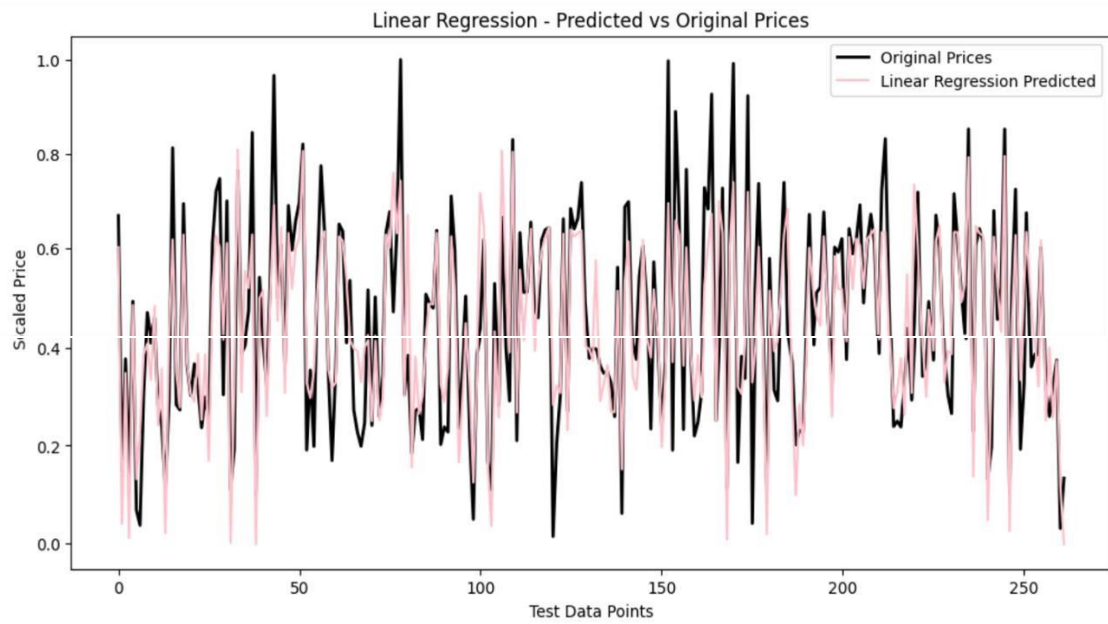


Fig. Linear Regression – Predicted vs Original Prices

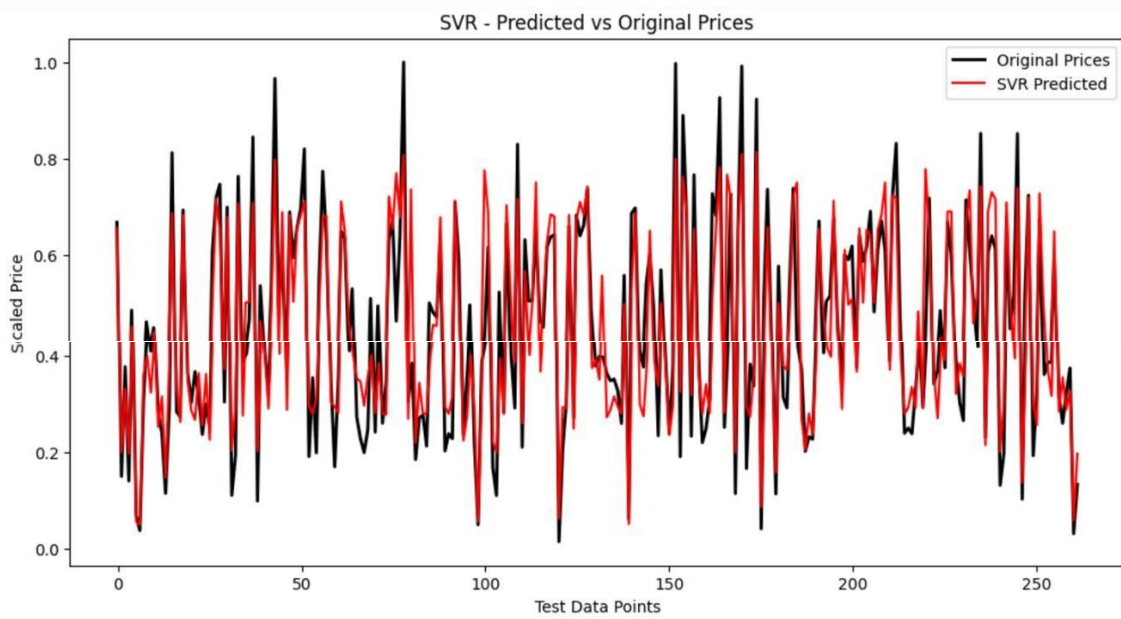


Fig. SVR – Predicted vs Original Prices

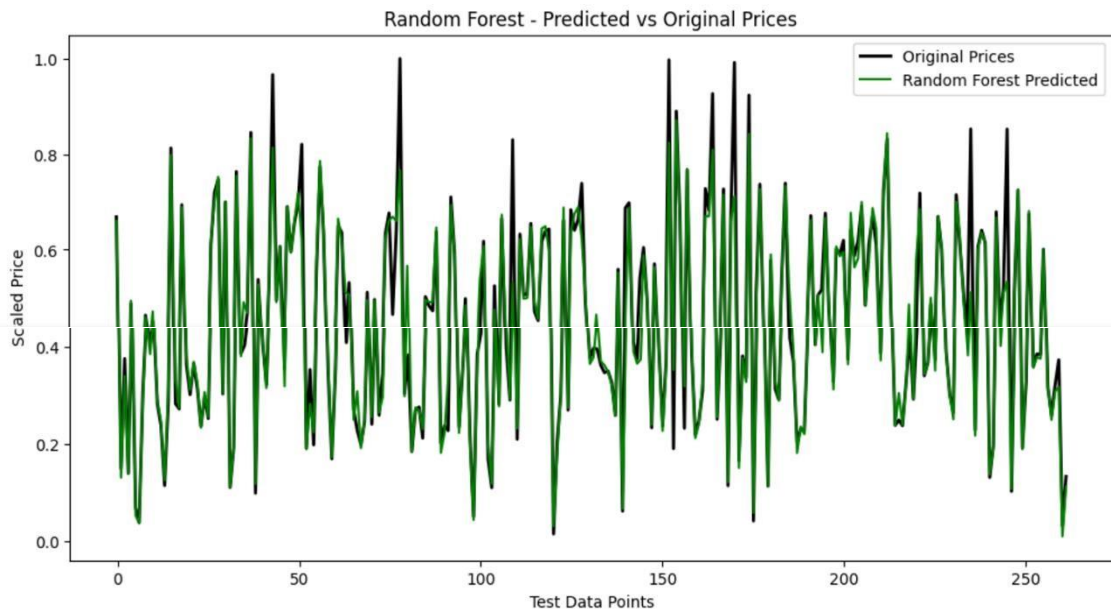


Fig. Random Forest – Predicted vs Original Prices

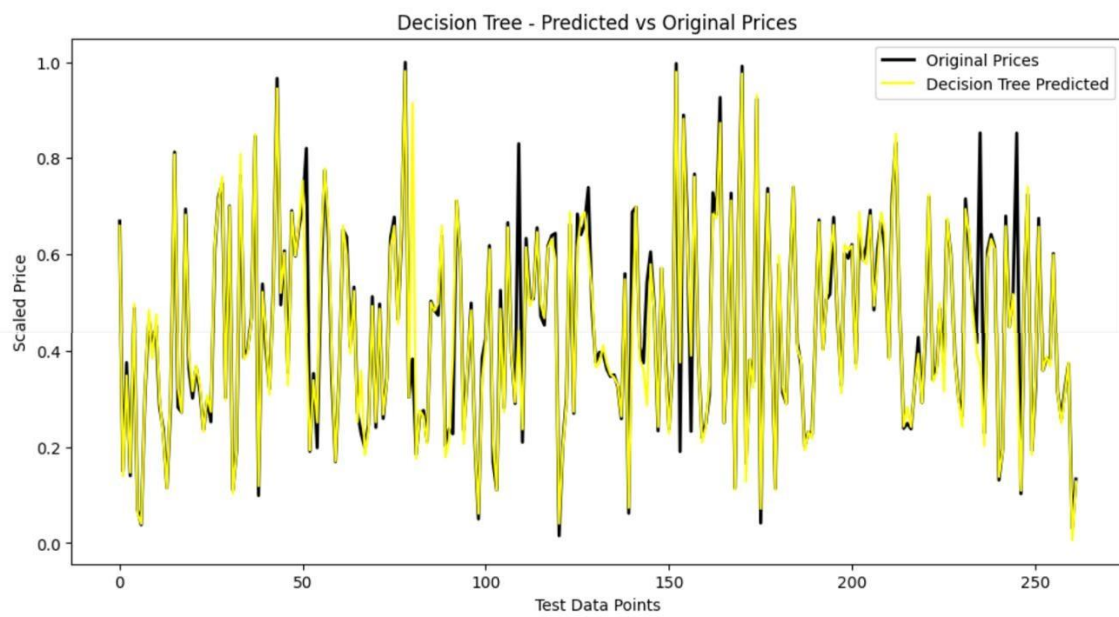


Fig. Decision Tree – Predicted vs Original Prices

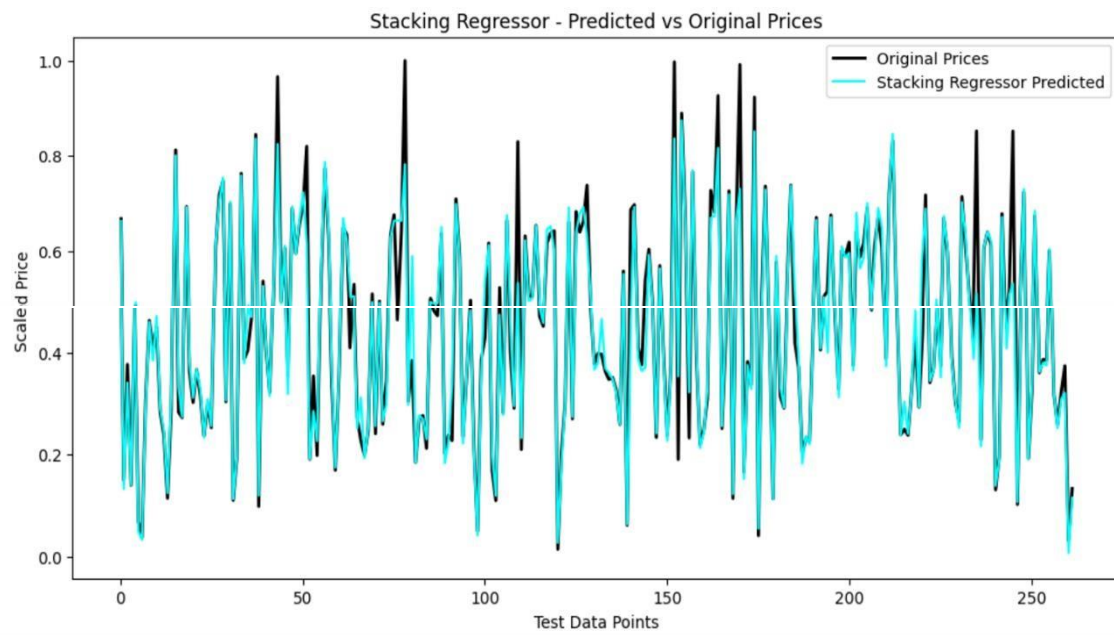


Fig. Stacking Regressor – Predicted vs Original Prices

APPENDIX 2

#Importing Libraries

```
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.linear_model import LinearRegression
from sklearn.svm import SVR
from sklearn.ensemble import RandomForestRegressor,
StackingRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

#Loading dataset from Yahoo Finance

```
ticker = "NFLX"
data = yf.download(ticker, start="2018-01-01", end="2024-01-01")
```

Displaying the first few rows

```
print("Loaded Stock Dataset:")
print(data.head())
```

#Showing the fetched dataset.

data

#Plotting the targeted attribute of the dataset.

```
plt.figure(figsize = (12,6))
```

```
plt.plot(data.Close)
```

```
plt.show()
```

Preprocessing and feature

engineering # Use 'Close' price for

prediction data = data[['Close']]

Feature Engineering: Add Moving Averages as features

```
data['100_MA'] = data['Close'].rolling(window=100).mean()
```

```
data['200_MA'] = data['Close'].rolling(window=200).mean()
```

#Displaying the data for the 100 days moving average.

```
data['100_MA']
```

#Plotting 100 days moving average.

```
plt.figure(figsize = (12,6))
```

```
plt.plot(data.Close)
```

```
plt.plot(data['100_MA'], 'r')
```

#Displaying the data for the 100 moving average.

```
data['200_MA']
```

```

#Plotting 100 and 200 days moving average.

plt.figure(figsize = (12,6))
plt.plot(data.Close)
plt.plot(data['100_MA'], 'r')
plt.plot(data['200_MA'], 'g')


# Drop rows with NaN values (initial rows due to rolling mean)
data.dropna(inplace=True)


# Prepare the features (X) and labels (y)

X = data[['100_MA', '200_MA']] # Features: 100-day and 200-day moving averages
y = data['Close'] # Label: Close price


# Scale the features and labels
scaler = MinMaxScaler()
scaled_features = scaler.fit_transform(X) scaled_labels =
scaler.fit_transform(y.values.reshape(-1, 1))


# Split dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(scaled_features, scaled_labels,
test_size=0.2, random_state=42)


# Define and expand individual models def
linear_regression(X_train, y_train, X_test):
model = LinearRegression()
model.fit(X_train, y_train) predictions =
model.predict(X_test) return model,
predictions

```

```
def svr_model(X_train, y_train, X_test):
    model = SVR(kernel='rbf')
    model.fit(X_train, y_train.ravel())
    predictions = model.predict(X_test)
    return model, predictions
```

```
def random_forest(X_train, y_train, X_test):    model =
RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train.ravel())    predictions = model.predict(X_test)

    return model, predictions
```

```
#def knn_regressor(X_train, y_train, X_test):

    # model = KNeighborsRegressor(n_neighbors=5)

    # model.fit(X_train, y_train.ravel())

    # predictions = model.predict(X_test)

    # return model, predictions
```

```
def decision_tree(X_train, y_train, X_test):    model
= DecisionTreeRegressor(random_state=42)
model.fit(X_train, y_train.ravel())    predictions =
model.predict(X_test)    return model, predictions
```

```

# Train and evaluate each model
models = {
    "Linear Regression": linear_regression,

    "SVR": svr_model,

    "Random Forest": random_forest,

    # "K-NN Regression": knn_regressor,

    "Decision Tree": decision_tree,
}

results = {}

for name, func in models.items():
    model,
    predictions = func(X_train, y_train, X_test)

    # Calculate metrics
    mse =
    mean_squared_error(y_test, predictions)
    mae =
    mean_absolute_error(y_test, predictions)
    rmse
    = np.sqrt(mse)
    r2 = r2_score(y_test,
    predictions)

    # Store results
    results[name] = {
        "Model": model,

        "Predictions": predictions,

        "MSE": mse,

        "MAE": mae,

```

```

    "RMSE": rmse,

    "R²": r2,

}

# Print metrics for each model
print(f"{name}:")    print(f"  MSE:
{mse:.4f}")          print(f"  MAE:
{mae:.4f}")          print(f"  RMSE:
{rmse:.4f}")          print(f"  R²
(Accuracy): {r2:.4f}\n")

# Stacking Regressor
stacking_regressor =
StackingRegressor(  estimators=[
    ("Linear Regression", results["Linear Regression"]["Model"]),

    ("SVR", results["SVR"]["Model"]),

    ("Random Forest", results["Random Forest"]["Model"]),

    # ("K-NN Regression", results["K-NN Regression"]["Model"]),

    ("Decision Tree", results["Decision Tree"]["Model"]),

],

    final_estimator=LinearRegression()

)

# Train the stacking regressor
stacking_regressor.fit(X_train, y_train.ravel())
y_pred_stacking = stacking_regressor.predict(X_test)
# Evaluate stacking regressor

```

```

mse_stacking = mean_squared_error(y_test, y_pred_stacking)
mae_stacking = mean_absolute_error(y_test,
y_pred_stacking) rmse_stacking = np.sqrt(mse_stacking)
r2_stacking = r2_score(y_test, y_pred_stacking)

```

```

# Add stacking results
results["Stacking"] = {
    "Model": stacking_regressor,

    "Predictions": y_pred_stacking,

    "MSE": mse_stacking,

    "MAE": mae_stacking,

    "RMSE": rmse_stacking,

    "R2": r2_stacking,

}

```

```

# Print stacking metrics print("Stacking:")
print(f" MSE: {mse_stacking:.4f}") print(f"
MAE: {mae_stacking:.4f}") print(f" RMSE:
{rmse_stacking:.4f}") print(f" R2
(Accuracy): {r2_stacking:.4f}\n")

```

```

# Plot predicted vs original prices

```

```

plt.figure(figsize=(14, 8))

```

```

plt.plot(y_test, label="Original Prices", color='black', linewidth=2)

```

```

for name, metrics in results.items():
    plt.plot(metrics["Predictions"], label=name, linestyle='-')

```

```
plt.title(f"{ticker} - Predicted vs Original  
Prices") plt.xlabel("Test Data Points")  
plt.ylabel("Scaled Price") plt.legend() plt.show()
```

```
# Plotting Predicted vs Original Prices for Linear Regression  
plt.figure(figsize=(12, 6))  
plt.plot(y_test, label="Original Prices", color='black', linewidth=2)  
  
plt.plot(results["Linear Regression"]["Predictions"], label="Linear  
Regression Predicted",color='pink', linestyle='-') plt.title("Linear Regression  
- Predicted vs Original Prices") plt.xlabel("Test Data Points")  
plt.ylabel("Scaled Price") plt.legend() plt.show()
```

```
# Plotting Predicted vs Original Prices for SVR  
plt.figure(figsize=(12, 6))  
plt.plot(y_test, label="Original Prices", color='black', linewidth=2)  
plt.plot(results["SVR"]["Predictions"], label="SVR Predicted",color='red', linestyle='-  
) plt.title("SVR - Predicted vs Original Prices") plt.xlabel("Test Data Points")  
plt.ylabel("Scaled Price") plt.legend()  
  
plt.show()
```

```
# Plotting Predicted vs Original Prices for Random Forest  
plt.figure(figsize=(12, 6))  
plt.plot(y_test, label="Original Prices", color='black', linewidth=2)  
  
plt.plot(results["Random Forest"]["Predictions"], label="Random  
Forest Predicted",color='green', linestyle='-') plt.title("Random Forest -  
Predicted vs Original Prices") plt.xlabel("Test Data Points")  
plt.ylabel("Scaled Price") plt.legend() plt.show()
```

```

# Plotting Predicted vs Original Prices for K-NN Regression

plt.figure(figsize=(12, 6))

plt.plot(y_test, label="Original Prices", color='black', linewidth=2)

plt.plot(results["K-NN Regression"]["Predictions"], label="K-NN Regression
Predicted",color='orange', linestyle='-')

plt.title("K-NN Regression - Predicted vs Original Prices")

plt.xlabel("Test Data Points")

plt.ylabel("Scaled Price")

plt.legend()

plt.show()

```

```

# Plotting Predicted vs Original Prices for Decision Tree Regression
plt.figure(figsize=(12, 6))
plt.plot(y_test, label="Original Prices", color='black', linewidth=2)

plt.plot(results["Decision Tree"]["Predictions"], label="Decision
Tree Predicted",color='yellow', linestyle='-') plt.title("Decision Tree -
Predicted vs Original Prices") plt.xlabel("Test Data Points")
plt.ylabel("Scaled
Price") plt.legend()
plt.show()

```

```

# Plotting Predicted vs Original Prices for Stacking Regressor
plt.figure(figsize=(12, 6))
plt.plot(y_test, label="Original Prices", color='black', linewidth=2)

plt.plot(results["Stacking"]["Predictions"], label="Stacking
Regressor Predicted",color='cyan', linestyle='-') plt.title("Stacking
Regressor - Predicted vs Original Prices") plt.xlabel("Test Data
Points") plt.ylabel("Scaled Price") plt.legend() plt.show()

```

```
#Showing the model who is most accurate. best_model_name =  
max(results, key=lambda x: results[x]["R2"]) best_model =  
results[best_model_name]  
print(f"The best model based on R2 (Accuracy) is: {best_model_name} with an R2 of  
{best_model['R2']:.4f}")
```